1. 反向代理 (Reverse Proxy)

定义: 服务端代理。Client → Proxy → Server (Client 不知 Server 存在)。

- **作用**: 负载均衡、隐藏后端结构、提高安全/可扩展性。
- **区别**: **正向**代理 (代理 Client) vs **反向**代理 (代理 Server)。

2. 系统尽量无状态 (Stateless)

A. Session 占内存: 高并发 (如 10w 用户) × KB 级 Session → 内存爆炸。⇒ 必须减少状态。**B. 安全**: Cookie/Session 易被窃取篡改，增加攻击面。**C. 维护成本**: 状态需存储/同步/失效处理 → 增加复杂度与开销。

D. Spring Bean 与状态:

- **Singleton**(默认): 多请求共享，含状态 → 线程不安全。
- **Prototype**: 每次请求新建，Controller/Service 可用 → 助逻辑无状态。

E. 为什么"状态重"?

- 对象需驻留内存、池化管理、序列化/反序列化。
- 池满 → LRU 淘汰 → 写盘 (Swap out) → 下次读回 (Swap in)。
- **结论**: 有状态 = 高频 Swap = 频繁 I/O = 性能杀手。

3. Instance Pool (类比 LRU Cache)

核心: 内存仅留有限实例，空间换性能，防 OOM。

- **流程**: 新请求 → 命中 (直接用) / 未命中 (磁盘加载)。
- **池满**: 按 LRU 淘汰最少使用实例。

4. 高并发下 DB 连接池问题

误区: × 用户多 = 连接多。✓ 连接数 ≠ 并发用户数。**为何不能太多?**

- 每个连接消耗 CPU/内存，可能阻塞在磁盘 I/O。
- **连接过多** → 线程上下文切换 (Context Switch) 剧增 → CPU 利用率 ↓ → 吞吐量差。

6. 设计原则 (Axiom)

Small pool, fully utilized.

- 连接池要小。
- **宁可线程等待连接，不要让数据库被线程淹没。**

---

1. 同步 (Synchronous)

Def: A 调 B, B 不回 A 阻塞。只等正常返回/抛异常。**问题 (必考)**:

- **A. 强耦合**: API 强绑定，接口改则全改。
- **B. 容错差**: 无缓冲/重试，网络抖动即报错。
- **C. 需描述**: 必须有 WSDL/API 文档。
- **D. 线程浪费**: 请求即占线程，高并发下线程耗尽。

2. 异步 (Asynchronous)

核心: A → Broker → B (类比微信，对方不在线也能发)。**优点**:

- **A. 解耦**: 只关注消息格式，不绑 API。
- **B. 高可靠**: 对方宕机消息暂存，不立败。
- **C. 削峰填谷**: 高峰积压，低峰处理。

缺点: 无法立即获结果；延迟变大 (多一跳)。

3. JMS (Java Message Service)

核心: 消息驱动，订阅 Queue/Topic → 触发逻辑。**Message 结构 (必背)**:

1. **Header**: 必须。固定字段 (MsgID, Timestamp)。
2. **Properties**: 可选。K-V 对，用于**过滤**。
3. **Body**: 可选。业务数据。

4. JMS 编程模型

流程: ConnectionFactory → Connection → Session → Producer/Consumer → send()/receive()。**异步方式 (重点)**:

- 注册 `MessageListener`。
- 中间件收信 → 回调 `onMessage()` 执行业务。

5. Queue vs Topic (必考)

- **Queue (P2P)**: 点对点。1 消息 → 1 消费者 (竞争)。类比: 公共邮箱。
- **Topic (Pub-Sub)**: 发布订阅。1 消息 → 所有订阅者。类比: 微信群。

6. 消息过滤 (Selector)

方式: 利用 Message **Properties** (SQL 语法)。**流程**: Prod 设属性 (type='tech') → Cons 设 Selector(type='tech') → 只收匹配消息。

7. Topic 离线收消息?

持久化订阅 (Durable Consumer):

- 消费者离线 → 消息不丢。
- 上线 → 一次性推送离线期间消息。

8. Kafka 基本原理

存储: 本质是**写文件 (Log)**。**特点**: Append-only (只能追加), Sequential Write (顺序写) → **极高吞吐**。

9. Kafka 消费模型

- **Offset**: 记录 Consumer 读到哪 (Commit)。
- **Consumer Group**: 同一 Topic 可由多组消费; **组内**协同 (每条消息只由组内一员处理) → 负载均衡。

10. Partition (分区)

Def: 1 Topic = 多个 Partition Log (物理拆分)。**好处**:

- **A. 并行消费**: 不同 Cons 处理不同 Partition → 吞吐 ↑。
- **B. 水平扩展**: 分区分布在不同机器/磁盘 → 破单机 IO 限制。

11. Replication (副本)

每个 Partition 有多副本，错开存储。Broker 间通过**心跳**检测故障 → 高容错。

12. 典型用法 (Async Req-Resp)

流程: Browser → Ctrl → **Topic1**(Req) → Listener → Service → DB → **Topic2**(Resp) → WebSocket → Browser。**削峰**: Topic1/2 作为 Buffer。**注意**: Topic 是日志，靠 **Retention/TTL** 删除过期消息 (类比 Redo Log)。

1. 异步系统"结果回推"问题

背景: 异步架构下，HTTP 请求结束，Client 只知"处理中"。但结果仍需回传。**解法**: WebSocket 是**结果回推 (Result Push)** 的核心方案。

2. 为何用并发容器 (Concurrent)

高并发长连接: 同时连接/断开/发消息 → 多线程读写会话集合。

- **Map**: 普通 Map 线程不安全 → 用 **ConcurrentHashMap** (支持高并发读写)。
- **计数 (在线人数)**: 普通 int 不安全 (count++) → 用 **AtomicInteger** (原子操作)。
- **结论 (必考)**: Endpoint 中, **会话集合 + 计数器**必须并发安全。

3. Ajax+Timer vs WebSocket

Ajax + Timer (轮询):

- **缺点**: 不知后端何时完。
- **定时太长**: 体验差; **太短**: 频率高，服务器压力大。
- **高并发**: 疯狂轮询 → 打爆后端。

WebSocket:

- **特点**: 长连接 Session, **全双工 (Full Duplex)**。
- **流程**: Client 建连坐等 ↔ Server 处理完直接 Push。
- **代价**: 前端需维护连接，有资源成本。

4. 消息非字符串? (Encoder/Decoder)

Encoder (编码器): 后端 Java Object → Text/String → 发给前端。**Decoder (解码器)**: 前端 Text → 后端解析 → Java Object (便业务处理)。

5. 多消息类型设计 (重要)

场景: 聊天 (A), 加入 (B), 通知 (C)...

- **Encoder**: 注册**多个**。每种类型一个 Encoder，根据对象类型自动匹配。
- **Decoder**: 通常**只写一个**。返回公共父类 `Message`。内部 Decode 时判断内容构造对应子类。

6. Ajax 异步 vs JMS/Kafka 异步 (必考)

Ajax 异步 (前端层面):

- **解决**: 请求慢阻塞页面。
- **效果**: 不刷新整页，局部更新，数据驱动 UI。
- **本质**: **提升交互体验**。

JMS/Kafka 异步 (后端层面):

- **背景**: 瞬时流量 (如订单) → 线程/DB 锁瓶颈 → 系统崩。
- **思路**: 请求写入 Queue(Topic) 立即返"处理中" → Listener 按能力消费。
- **本质**: **削峰填谷**, 保护后端系统。

1. 什么是事务 (TX)

定义: 一组操作绑定 → 原子操作。要么全成功，要么全失败。

2. 传播行为 (Propagation) (必考)

- **REQUIRED (默认)**: 有 TX 则加入，无则新开。
- **REQUIRES_NEW**: **无论如何新开**，挂起原 TX。场景: **日志、审计**。
- **NOT_SUPPORTED**: 挂起 TX, 以非 TX 运行。
- **SUPPORTS**: 有则加入，无则非 TX 跑。
- **MANDATORY**: 必须在 TX 中调，否则抛异常。
- **NEVER**: 必须**不在** TX 中，若有则抛异常。

3. 日志场景 (重点案例)

需求: 日志不可靠，但不能因日志失败回滚主业务。**解法**: 主业务 (TX1) + 日志 (TX2, **REQUIRES_NEW**)。**效果**: TX1 成败不依 TX2; TX2 挂不回滚 TX1。

4. 为何普通变量不能回滚?

JVM 内存变量不受 TX 管理器控制。TX 只管 DB/消息等资源 (有 undo/redo)。普通变量无恢复机制。

5. NoSQL 事务弱?

机制不完整 (缺完善的日志/Undo/Redo/恢复)，边界与回滚能力弱于传统 RDBMS。

7. 隔离级别 (低 → 高)

- **READ_UNCOMMITTED**: 读未提交 (脏读)。
- **READ_COMMITTED**: 读已提交 (RC)。无脏读，有不可重复读/幻读。
- **REPEATABLE_READ (RR)**: MySQL 默认。同 TX 多次读一致。仍可能有幻读。
- **SERIALIZABLE**: 串行化。最强隔离，性能最差。

8. 并发异常 (必考)

- **脏读 (Dirty Read)**: 读到对方未提交数据 → 对方回滚 → 数据不存在。
- **不可重复读**: 同一 TX 两次读同一行，结果不同 (中间有人 update)。
- **幻读 (Phantom)**: 记录没变，但多/少了行 (中间有人 insert/delete)。

9. 隔离级别权衡

高级别: 数据安全 ↑，并发 ↓。**低级别**: 并发 ↑，异常风险 (脏/幻) ↑。

10. 可重复读原理

同 TX 多次查结果一致。实现: **MVCC** (多版本并发控制) 或 **加锁**。

11. 分布式事务 (2PC)

Phase 1 (Prepare): 协调者问，参与者投 (一票否决)。**Phase 2 (Commit/Rollback)**: 全同意 → 提交; 任一反 → 回滚。**问题**: P2 网络挂 → 参与者猜结果 (Heuristic) → 不一致 → 需补偿。

12. 乐观锁 vs 悲观锁

- **乐观 (Optimistic)**: 假设冲突少。读不锁，**提交时查版本**。冲突则重试。
- **悲观 (Pessimistic)**: 假设冲突多。读即加锁，别人等待。

13. 粗粒度锁 (Coarse-grained)

场景: 改 Customer 时不许改关联 Address (1:N)。**乐观**: 共享 Version。改 Cust 或 Addr 都 Version+1。提交时 Version 不一致则败。**悲观**: 提交前对**共享对象**加锁。

14. 小项目为何不加 TX 也没事?

A: 出错手动删库模拟回滚。**I**: 并发小掩盖问题。高并发下问题才会暴露。

0. 事务与"事务性资源"

- **定义**: 对"事务性资源"的一组操作集合 (典型: 数据库数据)。
- **边界**: BEGIN / COMMIT / ROLLBACK。
- **语义**: 要么全部成功(COMMIT), 要么全部不生效(ROLLBACK)。
- **例**: 转账 (扣 A + 加 B) 必须不可分割，不能永久留下"扣了但没加"的中间态。

1. ACID 四特性 (数据库视角)

- **A 原子性 (Atomicity)**: 事务是不可再分工作单元。外部观察必须表现为: 整体成功或整体失败 (可回滚)。
- **C 一致性 (Consistency)**: 事务使数据库从合法状态 → 合法状态，前后都满足约束 (如余额 ≥0)。不仅是逻辑正确，也包含完整性约束。
- **I 隔离性 (Isolation)**: 并发执行时，每个事务看到的效果应像: 独占执行或按某个顺序串行执行。并发结果必须等价于某个串行顺序。
- **D 持久性 (Durability)**: COMMIT 成功后效果必须永久存在。崩溃恢复后必须回到"提交之后"的状态。

2. 并发异常: 读写/写写冲突

2.1 我写、别人读: 脏读 (Dirty Read)

- B 读到 A 未提交的中间值。
- 若 A 回滚，则 B 读到"不应存在"的值 (脏数据)。
- 对应最差隔离: Read Uncommitted。

2.2 我读、别人写: 不可重复读 (Non-repeatable Read)

- 同一事务内两次读同一行结果不同 (中间被别的事务提交更新)。
- **思路**: 读时"保护住"数据 (如加锁/MVCC)。

2.3 我写、别人也写: 写写覆盖/丢失更新 (Lost Update)

- 两事务都基于同一旧值更新，后写覆盖前写。结果不等价于任何合理串行顺序。
- **思路**: 写前加写锁。

3. 可串行化 (Serializable)

- **调度 (Schedule)**: 并发时各事务操作交错形成的全局顺序。
- **可交换规则**:
  - 读–读 (即使同一数据) 通常可交换。
  - 不同数据项上的操作可交换。
  - **不可随便交换**: 同一数据项且至少一个写。
- **定义**: 并发调度若等价于某个串行调度 → 可串行化。

4. 冲突可串行化 (Conflict Serializability)

- **冲突 (Conflict) 定义**: 来自不同事务; 作用在同一数据项; 且至少一个是写。
- **三类冲突**: $R-W$ (Ti 读 X, Tj 写 X); $W-R$; $W-W$。
- **直觉判定**: 若能通过不断交换"可交换"操作，变形为某个串行调度 → 冲突可串行化。

5. 优先图 (Precedence Graph)

- **构图**: 节点 =Ti。若调度中 Ti 与 Tj 冲突且 Ti 在前 → 加边 $Ti \to Tj$ (表示必须 Ti 在先)。
- **判定**: **无环** $\iff$ **冲突可串行化**。
- **有环**: ⇒ 不可冲突串行化 (互相要求"你在我前")。

6. 视图可串行化 (提及)

非冲突可串行化可能仍视图可串行化(验证复杂，工程不常用)。

7. 原子性 + 持久性: Undo/Redo 日志

7.1 为什么 COMMIT 后仍可能"丢"

- 修改在 Buffer Pool 形成脏页，写回磁盘通常异步。
- 若提交后仍在内存，随即崩溃 → 内存丢失 → 破坏持久性。

7.2 为什么不提交时强制刷所有脏页

- 数据页写盘是随机 IO，强制刷盘 IO 巨大、性能极差。
- 数据库倾向"攒一攒再批量落盘"。

7.3 日志为什么快

- 数据页: 随机写。
- **日志**: 顺序追加写 (append)。顺序 IO 远快于随机 IO。

8. Undo vs Redo: 功能与记录

8.1 Undo Log (支持回滚 → 原子性)

- **目标**: 撤销修改，回到事务开始前。
- **记录**: 旧值 `old_value`。典型 $< T, X, old_val >$。
- **用于**: 业务回滚、锁超时、崩溃恢复 (撤销未提交事务)。

8.2 Redo Log (支持重做 → 持久性)

- **目标**: 重放提交效果，回到提交后状态。
- **记录**: 新值 `new_value`。典型 $< T, X, new_val >$。
- **用于**: 已提交但未落盘时崩溃。

9. 故障类型与恢复目标

- 逻辑回滚/锁超时 → Undo。
- **系统崩溃**:
  - 已提交但未落盘 → Redo。
  - 未提交但已刷脏 → Undo。

10. 崩溃恢复的关键分类: 危险组合

- **危险 A**: Committed + Not Flushed → 持久性破坏 → **Redo**。
- **危险 B**: Active/Incomplete + Flushed → 原子性破坏 (磁盘含未提交结果) → **Undo**。
- **为何未完成也刷脏?** IO/CPU 并行提升吞吐; 大事务内存不够必须刷; LRU 淘汰触发。

11. Steal/No-Steal & Force/No-Force

11.1 Steal vs No-Steal (未提交能否落盘?)

- **No-Steal**: 不许。优: 原子性易; 缺:Buffer 压力大, 大事务差。
- **Steal**: **允许**。优: 释放 Buffer, 高并发。
- **代价**: 磁盘含未提交结果 → 崩溃须 Undo。
- **结论**: **Steal 需要 Undo 日志**。

11.2 Force vs No-Force (提交是否强制落盘?)

- **Force**: 强制。优: 持久性强; 缺: 频繁随机写, 性能差。
- **No-Force**: **不强制** (后台刷)。优: 性能好。
- **代价**: 提交后在内存 → 崩溃会丢 → 须 Redo。
- **结论**: **No-Force 需要 Redo 日志**。

11.3 组合: Steal + No-Force 性能最好，最常见。

13. WAL (预写日志)

- 必须先把日志写成功 (落盘)，才允许对应数据页落盘。
- **口诀**: 先日志，后数据。
- 推论: Steal → 刷页前保证 Undo Log 落盘; No-Force → Commit 时保证 Redo Log 落盘。

14. 日志类型

- **逻辑日志 (Logical)**: 记 SQL。优: 小。缺: 恢复需解析, 慢, 可能不安全 (如工资 *1.05)。
- **物理日志 (Physical)**: 记 Page+Offset+Value。优: 恢复快, 幂等。缺: 结构变化难逆推。
- **物理-逻辑日志**: 物理定位 Page + 逻辑记对象。

15. 为什么 Redo 偏物理、Undo 偏逻辑

- **幂等性 (Idempotence)**: 恢复可能多次执行结果必须不变。物理日志满足; 逻辑日志不一定。→ **Redo 偏物理**。
- **失败可重做性**: 恢复中再次失败必须能安全重放 → 物理更稳。
- **可逆性 (Reversibility)**: Undo 要"反向回去"。逻辑日志 (Insert ↔ Delete) 易表达; 纯物理在页分裂下难逆推。→ **Undo 偏逻辑**。

16. 恢复流程（Undo/Redo）

- **Undo 恢复**：找有 START 无 COMMIT 的事务 → 按 Undo(old) 撤销 → 记 ABORT。
- **Redo 恢复**：找已 COMMIT 事务 → 按 Redo(new) 重放。
- **Checkpoint**：给“磁盘一致性基准点”，避免从头扫日志。

1. Runnable 的好处

- **单继承限制**：Java 单继承，继承 `Thread` 就不能继承别的类。
- **接口灵活**：`Runnable` 是接口，可实现多个。
- **解耦**：只需实现 `run()`。符合“**任务（逻辑）与线程（执行载体）分离**”思想。

2. synchronized（监视器锁）

- 每个对象自带一把内部锁：**Monitor / Intrinsic Lock**。
- 进入 synchronized 方法/块前必须先拿锁。拿不到 → **阻塞等待**。
- **互斥**：同一时刻只有一个线程持有该锁。

3. 可重入锁（Reentrant）解决什么

- **场景**：递归/嵌套调用中，同一线程再次请求同一把锁。
- **问题**：若不可重入 → 自己把自己锁死（**自死锁**）。
- **Java 实现**：`synchronized` 是可重入的。记录重入次数，退出最外层才真正释放。

4. long / double 原子性问题

- Java（历史/平台相关）可能不保证 64 位读写原子性（读到“撕裂值”）。
- **解决（笔记强调）**：使用 `volatile`。
  - 用于多线程场景的**可见性**。
  - 提供一定程度的读写一致性保障。
- **记忆点**：并发需考虑原子性 + 可见性 + 有序性（常结合 CAS 理解）。

5. 活锁（Livelock）

- **定义**：线程无阻塞、未退出，一直在运行但**无进展**。
- **典型**：互相“礼让”（你给 A 我回 B ↔ 你收 B 又回 A）。
- **结果**：CPU 资源被消耗，系统不推进。

6. 自旋锁（Spin Lock）

- **缺点**：一直循环等待 → **CPU 空转**，浪费时间片。
- **优点**：避免**线程上下文切换**（Context Switch）。
- **适用**：无需保存/恢复寄存器/栈，若**等待时间极短**，自旋比阻塞/唤醒更快。

7. 不可变对象（Immutable Object）

- **定义**：创建后内部状态永不改变。
- **好处**：并发下**天然安全**（无竞态风险）；通常不需要 `synchronized`；逻辑清晰。
- **代价**：创建更多对象 → 内存占用与 GC 压力增大。

8. Java Lock 类思想（笔记重点）

- 倾向于“**拿锁要么一次拿全，要么放弃**”。
- 不允许“拿着一把锁去等另一把锁”（避免死锁模式）。
- 拿不全就释放（体现为 tryLock / 可中断 / 超时等机制）。

9. 虚拟线程（Virtual Thread）

- **背景**：大量短小任务（HTTP/JDBC）。若 1 任务 =1 平台线程 → 创建切换成本高，资源浪费。
- **原理**：用大量 Java 层轻量线程 **复用**少量 OS 平台线程（M:N 模型）。
- **机制**：当虚拟线程阻塞（Block）→ 让出平台线程（Yield）→ 平台线程跑别的任务。
- **效果**：大幅提升吞吐与并发规模。

1. 为什么需要缓存？想解决什么问题？

典型访问链路（Cache-Aside 思路）：

- 先查缓存（内存快）；
- miss 再查 DB（磁盘 IO + 进程间通信 + SQL 执行 + 结果集 + ORM 映射）；
- 查到后写回缓存；
- 下次直接命中。

核心目的：减少频繁访问数据库次数，提升读性能，减轻 DB 压力。**DB 慢的原因（应用视角）**：

- 数据最终落盘：磁盘读写。
- 应用 ↔DB：进程间通信。
- SQL 执行、返回、ORM 映射/组装对象成本。

2. DB 也有缓存，为什么还要应用缓存？

DB 缓存（如 Buffer Pool 等）：

- 策略由 DB 决定。
- 你难以精确控制“哪些一定常驻、不淘汰”。
- 通常是页/块/执行结果层面，不是业务对象层面。

应用缓存（Redis/Memcached/Spring Cache）：

- 你决定缓存什么对象（Book/User/Department…）。
- 你决定 key、TTL、淘汰策略。
- 可按业务建模：把缓存当业务层的一部分设计。

结论：DB 缓存你管不了；应用缓存你可控、可按业务设计。

3. 哪些数据适合缓存？哪些不适合？

3.1 适合缓存：读多写少、变化慢（read-heavy）

- 例：书籍基础信息、院系列表、城市/国家列表、商品类目。
- 命中缓存可显著减少 DB 访问。

3.2 不适合缓存：写频繁、强一致性敏感

- 例：订单、库存、余额、交易流水。
- **原因**：频繁写会引发一致性复杂度：
  - DB 与缓存同步更新（双写一致性）。
  - 要么一起成功要么一起失败，可能需要事务/MQ/一致性策略。

课程原则：经常写的对象不要缓存；经常读且基本不变才缓存。

4. 缓存会倒逼数据库设计（拆表/重构）

- **例**：Book 基础信息适合缓存，但库存不适合。
- 若 Book 表混在一起（基础信息 + 库存字段）：库存频繁更新 → 缓存 Book 难/不稳定。
- **正确做法（拆表）**：
  - Book：只放基本不变字段。
  - Stock：book_id + stock_count（高频更新）。
- **结论**：缓存策略会反向影响数据模型设计。

5. Spring Cache（进程内缓存）演示要点

- 开启：`@EnableCaching`；标注缓存方法：`@Cacheable`。
- 指定缓存命名空间/前缀（如”books”）。
- **为什么要 namespace？** KV 存储 key 会冲突：Book id=1、User id=1 都是“1”。典型 key：`books:1`、`users:1`。本质：用前缀隔离不同对象域。
- **性能现象**：
  - 不开缓存：每次走 DB → 慢且波动。
  - 开缓存：前几次慢（回源 DB + 写入缓存），后面几乎瞬间（命中缓存）。
  - 原因：省掉进程间通信 + SQL + ORM。

6. 缓存是独立进程为何仍快？

- 缓存是 KV，定位快（常见哈希结构，接近 O(1)）。
- 缓存中常存“已组装好的对象表示”（或可快速反序列化）。
- DB 回源链路更重：磁盘/SQL/结果集/ORM。
- 即使都有 IPC：缓存通常仍远快于 DB；还能显著减轻 DB 压力，让 DB 专注写与一致性。

7. Memcached 两个核心设计问题（必考）

7.1 对象大小不一：内存怎么组织？

- 若紧密排列：需要记录偏移/元数据，管理复杂、查找慢、metadata 膨胀。
- **Memcached：Slab 分配**
  - 内存切成 slab，slab 内是固定大小 chunk。
  - 不同 slab 对应不同 chunk size（88B/112B/144B…）。
  - 对象来时：选“能装下的最小 chunk”。
- **取舍**：浪费一点空间换管理/访问速度与简单性。

7.2 分布式：key 放哪台？节点增减怎么办？

- **朴素 hash**：$hash(key)\%N$。N 改变（扩容/缩容）→ 几乎所有 key 映射变化 → 全量失效。
- **一致性哈希（Consistent Hashing）**
  - key 也 hash，node 也 hash。映射到环（$0 \ldots 2^{32}$）。
  - **存放规则**：顺时针遇到的第一个节点（最近节点）。
  - **增删节点**：只迁移被覆盖区间数据，绝大多数不动。
- 补充：Chord 等会用路由/指针表加速查找。

8. Redis：为什么更常用？

Redis 也是内存 KV，但更像“内存数据结构服务器”（比 Memcached 更“丰富”）。

9. Redis 缓存典型写法：Cache-Aside

以 person/user 为例（RedisTemplate 思路）：**读流程**：

1. Redis GET：key=user:{id}（通常存 JSON）。
2. **miss**：查 DB repository → 序列化对象为 JSON → Redis SET 写回 → 返回对象。
3. **hit**：反序列化 JSON → 对象 → 返回对象。

10. 对象多时代码重复 → 统一网关

- 若 50 张表/50 类对象，每个都写：Redis get → miss 查 DB → 写回 → 序列化 → 大量重复代码。
- **方案：抽象统一网关（Data Access Gateway）**
  - 上层只调用：`gateway.findOne(type, id)`。
  - gateway 内部决定来源：Redis / MySQL / 文件 / NoSQL …
  - 屏蔽细节：key 拼接、miss 回源、序列化等。
- **意义**：统一接口、屏蔽数据来源、减少重复。

11. Redis 可靠性与崩溃影响

- **应用崩溃/重启**：Redis 独立进程，通常不受影响；应用重启后缓存大概率仍在。
- **Redis 崩溃**：
  - 对应用表现：大量 cache miss → 回源 DB → 性能下降。
  - 功能通常仍可用（回源后再写回）。
  - Redis 常有持久化/刷盘机制，重启后可能恢复部分数据。

12. Redis 也能做消息中间件

- Redis 可作为 broker（发布/订阅）。
- **Spring 典型结构**：
  - `Listener`：订阅某 topic/channel。
  - `Receiver`：类似 onMessage，定义收到消息后的处理逻辑。
  - `ListenerAdapter`：把消息到来映射到 Receiver 方法。
- **发送侧**：Browser → Controller → RedisTemplate publish → topic。
- **链路**：HTTP 请求 → controller 发布 → listener 订阅 → receiver 处理。
- **结论**：Redis 不仅能缓存，也能做简单消息系统（demo 级）。

1. 全文搜索的需求（Full-text Search）

- **输入**：大量文本文件（可能非常多）。
- **目标**：给定查询词/短语，系统能回答：
  - 出现在哪些文档（doc）；
  - 在文档什么位置（行/词序号/偏移量）。
- **本质**：在海量非结构化文本中快速定位 + 返回命中位置 + 排序。

2. 反向索引 / 倒排索引（Inverted Index）

传统索引（数据库类比）：

- 从“文档/记录”出发 → 映射到字段/存储位置。

倒排索引：

- 从“**词（term）**”出发 → 指向它出现的：
  - 文档集合（doc list）；
  - 文档内位置（positions/offsets）。
- **一句话**：term → postings（出现在哪些 doc、出现多少次、在哪些位置）。

3. Lucene 全文搜索基本流程

1. 提供原始数据（text/HTML/PDF/Word…）；
2. 建立索引（indexing）；
3. 用户发起查询（query）；
4. 在索引中搜索并返回结果（包括命中文档、命中位置等信息，取决于索引配置）。

4. 索引的“域”（Field）是什么

- 文档在索引里由多个 Field 组成（类似结构化的“列”）。
- **索引结构可粗略理解为三类信息**：
  1. Field 的定义与统计（有哪些 Field、取值分布/出现次数等）；
  2. term → doc 的映射（某词出现在哪些文档）；
  3. doc 内位置信息（该词在文档中出现次数、具体 positions/offsets）。

5. 排序问题：TF-IDF（Ranking 核心）

搜索不仅要“找得到”，还要“排得对”。

- **TF（Term Frequency）**：词在该文档出现越多 → 通常越相关。
- **IDF（Inverse Document Frequency）**：词在越多文档出现 → 区分度越低 → 权重越小。
- **结论**：排序不是只看出现次数，还要看词的区分度。

6. 准确率 & 召回率

- **准确率（Precision）**：返回结果里，相关的占多少？“我给你的是否大多是对的”。
- **召回率（Recall）**：所有真正相关的结果里，你找回了多少？“该找回的我找回了多少”。
- **常见权衡**：提高召回率可能降低准确率，反之亦然。

7. KNN（近邻/分类）在 Lucene 场景

- 把每本书表示为向量（主题/风格/情绪/类型…特征维）。
- **向量空间**：距离越近 → 语义越相近。
- **对新书分类**：
  1. 找最近的 k 个邻居（k 常取奇数）；
  2. 看邻居里哪个类别最多；
  3. 多数类即预测类别。
- **记忆点**：KNN = “看你周围的人是谁”。

8. Field 要不要存储？

Field 分两类策略：

- **Stored**：字段值存入索引 → 命中后可直接返回（适合短字段，如 author/title）。
- **Not Stored**：不存字段值 → 索引更小、更快；展示时回源读取。

正文 content 若全部存入索引：

- 索引体积显著增大，性能与存储成本上升。

常见做法：

- 索引只保存倒排结构 + 必要元信息。
- 正文不存，只存定位信息（如 path/url/docId 指针）。
- 展示时再根据指针回源读取原文件。

0. 有状态 vs 无状态（REST 核心）

× 有状态（非 REST）

- 登录后服务器内存保存 session（sessionId → user）。
- 后续请求依赖 cookie（sessionId）。
- **问题**：服务器必须记住“你是谁”；请求被转发到另一台机器 → session 丢失；需要 sticky session 或 session 共享 → 架构复杂、不易扩展。

✓ 无状态（REST）

- 每个请求都携带完整身份信息（Authorization: Bearer <JWT> / OAuth Token / API Key）。
- **服务器**：只根据当前请求认证，不依赖历史请求。
- **优点**：易于负载均衡，易扩展，更符合 Web 架构。

1. 为什么需要 Web Service？

- **背景**：后端 Service 需被不同语言/平台（Web 前端/Android/Python）调用。单纯 Java 方法调用不可能跨语言。
- **思路**：统一用 HTTP 作为入口。引入 Web Service：语言无关、平台无关。

2. Web Service 的含义

- **Web**：基于 Web 协议（通常 HTTP），穿透能力强（比 JMS/Kafka 强）。
- **Service**：与编程语言无关，跨平台调用。

3. SOAP Web Service

3.1 SOAP 是什么？

- SOAP = Simple Object Access Protocol。
- 基于 XML 的远程对象调用协议。本质：XML 文本。

3.2 WSDL 是什么？

- WSDL = Web Service Description Language。
- Web Service 的 **说明书 / 接口文档**。一个 WSDL 描述完整服务能力。
- **主要组成**：
  - **Message**：请求/响应的数据结构。
  - **PortType**：接口定义（有哪些方法）。
  - **Binding**：接口 + 协议（HTTP / SOAP）。
  - **Service**：服务访问地址（URL）。

3.3 SOAP 调用流程

1. 客户端获取 WSDL。
2. 使用工具（如 Maven 插件）生成：接口、**Stub（代理类）**。
3. 客户端调用接口方法：
   - Stub：**marshal**（对象 → XML）→ 发送 SOAP 请求。
   - 服务端：**unmarshal**（XML → 对象）→ 调用真实 Service → 返回 SOAP 响应。

3.4 SOAP 的缺点

- XML 冗余大、性能差；WSDL 复杂、学习成本高；强接口耦合；序列化/反序列化开销大。
- → 逐渐被 REST 取代。

4. REST / RESTful Web Service

4.1 @RestController 是什么？

- 用于构建 REST 风格接口。返回值通常是：JSON / XML（不是页面）。

4.2 REST 的核心思想（三点）

1. **Representation（表述）**：URI 定位资源；HTTP 方法表示操作；Body 为资源表述（通常 JSON）。
2. **State（状态）**：服务端无状态；客户端维护状态（token / 页面状态）。
3. **Transfer（转移）**：客户端拿到资源 → 页面/客户端状态发生变化。即：状态转移。

5. REST ≠ 随便写 Controller

× 不严格 RESTful（函数式）

- `/book/findById`, `/book/placeOrder`
- URL 暴露“动作/函数”。

✓ 严格 RESTful（资源导向）

- **URL**：定位资源；**HTTP 方法**：表达动作。
- **典型设计**：
  - `GET /books/1`（查询）
  - `POST /books`（新增）
  - `PUT /books/1`（修改）
  - `DELETE /books/1`（删除）
- **规则总结**：同一资源 → 同一路径；动作靠 HTTP 方法区分；URL 中最多出现 id / query 参数。

1. 单体应用（Monolith）的问题

- **强耦合、强依赖**：模块间相互依赖，一个出问题可能拖垮整体。
- **单点失败**：任一模块崩溃，整个系统不可用。
- **资源无法差异化调优**：所有模块共享同一套 CPU/内存/线程/连接池。无法只给“热点模块”单独扩容。

2. 微服务的核心思想

- 把一个巨大的单体应用 → 拆成多个**小而独立的服务**。

- 每个服务：独立部署、独立运行、独立扩缩容。
- **主要好处**：
  - **故障隔离**：一个服务挂了，不影响其他服务。
  - **资源独立扩缩容**：热点服务单独加机器，冷门服务不用浪费资源。

3. 微服务的代价与权衡（Trade-off）

- 模块调用不再是：进程内方法调用；而是：**跨进程 / 跨网络调用**（HTTP/RPC/MQ）。
- **后果**：性能必然下降；系统复杂度上升。
- **换来的好处**：复用性显著提升；可直接调用第三方服务（只看接口，不看源码，如第三方认证）。

4. 负载均衡（Load Balancing）

- **常见策略**：Round Robin（轮询，默认）、Random（随机）、Least Connections（最小连接数）、Weight（加权）。
- **轮询前提**：服务必须是 **无状态（Stateless）**。否则 session 会丢失（请求打到不同实例）。

5. Eureka：注册中心（Registry）

- **作用**：维护 ServiceName → ServiceInstanceList；管理服务生命周期。
- **服务启动**：向 Registry 注册 → 定期发送心跳（Renew）→ 超时未续约则自动剔除。
- **调用过程**：客户端查 Registry → 返回可用实例 → 客户端调用。
- → **Registry 本质是 Key-Value 存储**。

6. Registry 中的 Value 到底是什么？

- Value 本质不是"方法列表"，而是一个 **对象引用（Object Reference）**。
- 内部包含：IP / 端口 / 协议等定位信息。
- Registry 只负责一件事：告诉如何找到服务实例。不关心有哪些方法、业务逻辑。
- **类比 JNDI**：Key=jdbc/bookstore, Value=DataSource 的远程资源引用。

7. 为什么只有 Registry 不够？

- **问题**：客户端仍然要：知道服务名、自己查 Registry、自己写地址/负载均衡逻辑。
- **结果**：客户端依然很复杂。

8. Gateway（网关）的引入

- **目标**：客户端只访问一个入口（Gateway）。所有复杂度下沉到 Gateway。
- **Gateway 内部负责**：查 Registry、获取实例引用、负载均衡、请求转发。

9. Gateway 能解决的痛点

- **解耦端口**：服务端口变了，客户端不用改代码，只改 Gateway 配置。
- **但问题仍然存在**：Gateway 不能写死 IP/端口。

10. Gateway + Registry 才是完整解

完整流程：

1. 服务启动 → 注册到 Registry（Key: book-service, Value: IP/Port）。
2. **Gateway 路由规则**：不写具体地址，只写：转发到 `book-service`。
3. **Gateway**：从 Registry 拿实例 → 做负载均衡 → 转发请求。

11. 函数式服务（FaaS）

- 允许你写：`Function<T, R>`, `Supplier<R>`, `Consumer<T>`。
- 支持：map / transform（如 Flux）、组合（compose / andThen）。
- **重要澄清**：写成 Function ≠ 一定无状态。是否无状态取决于：是否依赖/修改共享状态。

12. 理想微服务架构（考试版）

- **客户端**：只访问 Gateway。
- **服务端三件套**：
  - **Registry**：名字 → 实例引用。
  - **Gateway**：路由 + 负载均衡 + 转发。
  - **Service**：启动注册 + 定期 Renew。

13. 遗留问题 & 为什么作业不拆数据库

1）微服务的根本代价

- 单体：进程内调用（快）。微服务：网络调用（慢）。
- **典型 trade-off**：可用性/隔离性/扩展性 ↑ vs. 性能/系统复杂度 ↑。

2）数据库拆分的现实问题

- **理论上**：每个微服务应有独立数据库。
- **现实中（尤其课程作业）**：本地机器难以同时跑多个 MySQL + 多个 Redis + 多个服务实例。
- **所以**：作业阶段 **不强制拆数据库**。

A. 索引（Index）

1）聚簇索引 Clustered Index（InnoDB 核心）

- **定义**：叶子节点直接存放数据行 → 索引顺序 = 数据顺序。
- **优点**：范围查询快，连续 IO 友好。
- **限制**：只能有一个（数据物理顺序唯一）。InnoDB 默认建在主键（PK）上。

2）聚簇索引键能不能用 name？

- **可行**：只要可比较/排序。
- **不建议（四大理由）**：
  - 不唯一 → 定位慢。
  - 会变化 → 改名 ≈ 整行搬家 + 二级索引连锁更新。
  - 键大更慢 → B+ 树层高 ↑，IO ↑。
  - 比较复杂 → Collation 成本 > 整数。

3）二级索引 Secondary Index（回表）

- 叶子存：索引列 + 主键值。
- **查询流程**：二级索引找 PK → 用 PK 去聚簇索引取整行 ⇒ **回表（Back to Clustered Index）**。
- **结论**：依赖聚簇索引；能覆盖就尽量覆盖（Covering Index）减少回表。

4）为什么用 B+Tree？

- 磁盘按页 IO。B+Tree 分叉大 → 树矮 → IO 少。二叉树高 → IO 爆炸。

5）其它类型

- **FULLTEXT**：倒排索引（TF-IDF）。
- **HASH**：等值快（O(1)），无范围/排序。
- **SPATIAL**：R-Tree（二维几何）。

6）小表要不要索引？

- 表很小 → 全表扫可能更快。索引有维护成本 ⇒ 小表谨慎建。

7）主键设计

- 越长 → 索引文件越大。复合主键 → 二级索引叶子带 PK 变大。
- **建议**：短、稳定、单列优先（如自增整数）。

8）拆表

- 热字段与冷字段分离 → 减少无效 IO ⇒ "少读磁盘"优先级高。

9）大字段与前缀索引

- 行逻辑上限 64KB。大字段可能溢出（存指针）。
- **前缀索引**：只索引前 N 字符 → 省空间。

10）行格式 ROW_FORMAT（取舍）

- **Dynamic**：大字段外置，行更稳定。
- **Compact**：尽量行内。
- **Compressed**：压缩省空间，CPU ↑。

11）复合索引 & 最左前缀（必考）

- Index: `(last, first)`
- **可用**：`last='Z'`, `last='Z' AND first='S'`, `last='Z' AND first LIKE 'S%'`
- **不可用**：`first='S'`（跳过最左），`first='S' AND last='Z'`（本质仍需最左开始），`LIKE '%abc'`（前导通配）。
- **设计**：按高频 WHERE/排序顺序；索引多写变慢。

12）索引顺序：ASC/DESC 混合可用于优化 ORDER BY。

B. 表结构与数据类型（核心：减 IO）

1）类型能小就小 + NOT NULL

- 小类型 → 页存更多行 → 缓存命中 ↑。
- **NULL**：需额外标记，语义麻烦 ⇒ 尽量不用，用默认值。

2）范式（3NF/BCNF）

- 高范式（冗余少，Join 多）；低范式（查快，冗余多）。工程常折中。

3）数字 vs 字符串：ID 类数值型更省（4-8B vs 十几 B）。

4）VARCHAR vs BLOB：VARCHAR 优先行内；BLOB 易外置。

5）"打开的表"比预期多：元数据、并发/MVCC、缓存淘汰策略（LRU）。

6）临时表（性能坑）

- 触发：Order/Group/去重。占资源（FD/IO）。优化 SQL 减少触发。

7）上限决定者：OS FD 上限、文件系统限制、磁盘空间。

8）列数与行大小限制（必记）

- **列数**：≈ 4096。
- **行逻辑上限**：≈ **64KB**（65535B）。
- **行内物理上限**：受页大小（16KB）影响。单行不能太大（"半页级别"）。
- **溢出**：VARCHAR 可外置；**CHAR 不能溢出**（必须行内 → 易报错）。

C. InnoDB 运行与事务优化

1）OPTIMIZE TABLE：碎片整理。大表且稳定时有用。

2）autocommit：1 → 每条一事务 → 频繁刷盘。建议关掉，批量提交。

3）Buffer Pool：CPU 不满但慢 → IO 瓶颈（BP 太小）。

4）隔离级别与长事务：隔离强锁重。长事务可降级 RC/RR。

5）导入优化：

- 临时关外键/唯一检查。
- **批量**：`INSERT VALUES (...),(...)` 快于逐行。

6）AUTO_INCREMENT 高并发：按区间分配（ID 交错但唯一）。

7）落盘与刷脏：

- **Direct IO**：绕过 OS Cache。
- **脏页阈值**：小更平滑，大易卡顿。SSD 适随机 IO，HDD 适顺序。

8）操作级建议：

- TRUNCATE > DELETE（重置结构）。
- **别改主键**：改 PK ≈ 重建整表（聚簇索引）。

D. Buffer Pool（BP）细节

1）多实例：减少高并发锁竞争。

2）改造版 LRU（防污染）

- 纯 LRU 大扫描会挤掉热页。
- **InnoDB**：新页不放最热，放 LRU 中间（约 3/8 处）。

3）预读（Read Ahead）：线性（利用局部性）/ 随机。

4）刷脏（Cleaner）：阈值触发优于定时（更平滑）。

5）BP 预热：重启前 Dump 元信息，启动 Load → 快速恢复命中。

6）SQL 编译缓存：缓存执行计划（Plan），减少 parse/optimize 开销。

A. 为什么"日志"和"数据"不一致？

WAL（Write-Ahead Logging）

- **规则**：任何修改，必须**先写日志，再改数据页**。
- **出现"看起来不一致"的正常状态**：
  - 日志已写入磁盘，数据页还在内存。
  - 数据页写了一部分，事务没提交。
  - 事务已提交，数据页未刷盘。
- **结论**：这是为性能与崩溃恢复做的设计，不是错误。

B. 物理备份 vs 逻辑备份（必考对比）

1）物理备份（Physical Backup）

- **做法**：直接拷贝二进制文件。
- **优**：速度快（大库首选），恢复快。
- **缺**：兼容性差（同版本 MySQL）。
- **关键要求**：**FLUSH**（刷脏页）；**加读锁**（防数据变化，保一致性）。

2）逻辑备份（Logical Backup）

- **做法**：导出 SQL（mysqldump）。
- **优**：可移植性强（跨版本/迁移）。
- **缺**：慢，脚本大，恢复需逐条执行。
- **一致性**：放在单个事务中导出 → 要么全成功，要么全失败。

C. 全量 vs 增量

- **全量（Full）**：某一时刻全部数据（天/周）。
- **增量（Incremental）**：只备"变化部分"（依赖 **binlog**）。
- **策略**：周期性全量 + 持续 binlog 增量。

D. 在线（Online）vs 离线（Offline）

- **在线**：不停机（锁/MVCC）。优：业务不中断。代价：复杂，性能降。
- **离线**：停库再备。优：简单/快/一致性好。缺：业务中断。

E. 本地 vs 远程（工程原则）

- **黄金原则（必背）**：**备份不要和原数据在同一块盘上**。
- **最佳实践**：异盘、异机、异地域（防火灾）。

F. 导出格式：SQL vs CSV

- **SQL**：含结构 + 数据。可直接恢复。最常用。
- **CSV**：面向分析。易处理。不含结构（需先建表再导）。

G. 运维常识

- **压缩/加密**：省空间 / 防泄露。
- **调度**：定期全量 + 持续 binlog + FLUSH LOGS + 清理旧日志。
- **用途**：恢复、升级演练、测试环境、审计。

H. 备份不仅是"表数据"

需考虑 Trigger/Stored Procedure/Event。

- **兼容性**：跨版本/跨库差。
- **默认策略**：
  - ✓ **Trigger**：通常要导（属数据一致性约束）。
  - × **存储过程/函数/事件**：通常不导（偏逻辑/优化）。

I. 逻辑错误恢复（误删表）

核心思路（必考流程）：

1. 用最近一次 **全量备份恢复**。
2. **顺序回放 binlog**：
   - 回放到误操作前 → 跳过误操作语句 → 继续回放。
3. **注意**：binlog 必须顺序回放，× **不能并行**。

J. 三类日志明确分工（高频）

- **redo log**：已提交未落盘 → 崩溃后 **重做**。
- **undo log**：未提交/已写部分 → 崩溃或回滚时 **撤销**。
- **binlog**：所有逻辑变更 → **增量恢复 / 主从复制**。

K. 从单机到集群

Primary（写）/ Secondary（读）

- **好处**：冗余（容灾）+ 读性能提升。
- **实践**：轮换读流量，避免备机长期"**太冷**"（否则一切换就性能抖动）。

A. 分区是为了解决什么问题？

1）数据量太大，单表/单文件扛不住

- 单文件过大，单磁盘压力大。
- **思想**：把一张逻辑表按规则"切开" → 多个物理分区（可放不同文件/磁盘）。逻辑上仍是一张表。

2）数据量太大，查询效率下降

- 超大表扫描成本高。
- **关键优化：分区裁剪（Partition Pruning）**。SQL WHERE 命中分区键 → 优化器只访相关分区（跳过无关）→ IO 大幅减少。

B. 水平 vs 垂直（概念对比）

- **水平分区（Horizontal）**：按行切。行分布到多分区（时间/范围/哈希）。MySQL 分区主要就是水平。
- **垂直分区（Vertical）**：按列切。常用列/大字段拆表。本质是多表设计 + Join。MySQL 靠表设计实现。
- **补充**：行存（MySQL）偏水平；列存/NoSQL 偏垂直。

C. 行存 vs 列存

- **行存（Row）**：整行连续。OLTP/高并发/小事务/点查。MySQL 主打。
- **列存（Column）**：整列连续。OLAP/大扫描/聚合（SUM/COUNT）。
- **HTAP**：行存 + 分区 + 外部 OLAP。

D. 分区带来的收益（考试重点）

1. **存储与扩展**：分布到不同文件/磁盘/节点，单分区更可控。
2. **批量维护快**：`DROP/TRUNCATE PARTITION`。秒删历史数据，整批装载（比逐行 Insert 快）。
3. **查询优化**：分区裁剪 → IO 显著下降。

E. MySQL 四大分区类型（必背）

- **1）RANGE（范围）**：按连续区间（数值/日期）。
  - 语法：`VALUES LESS THAN`，常配 `MAXVALUE` **兜底**。
  - 支持表达式：`YEAR(date)`。TIMESTAMP 需转整数。
- **2）LIST（枚举）**：按离散值集合。
  - 语法：`VALUES IN`。** 无兜底 **（未命中报错）。
  - 缓解：`INSERT IGNORE`。
- **3）HASH**：目标均匀分布/防热点。变体：LINEAR HASH。
- **4）KEY**：系统哈希（PK/Unique Key）。变体：LINEAR KEY。

F. RANGE/LIST COLUMNS（多列）

- **比较规则**：元组比较（先比 c1 再比 c2，类似字符串）。
- **主要限制（必考）**：
  - 只能写列名，× **不允许函数**（如 `YEAR()`）。
  - **类型**：✓ 整型/日期/字符串；× TEXT/BLOB；△ Float（精度坑）。
- **规则**：边界必须单调递增。

G. 子分区（Subpartition）

- **含义**：先分区再子分区（二级）。例：年（Range）+ Hash。
- **物理数**：分区 × 子分区。
- **用途**：控制单分区大小，均匀分布，多维度组合（时间 + 用户）。

H. NULL 的行为（高频）

- **RANGE**：永远进 **最低分区**。删最低分区 → NULL 一起删。
- **LIST**：必须显式写 `VALUES IN (NULL)`，否则插失败。
- **HASH/KEY**：视为 0（常与 0 落同一分区）。

I. 分区 ≠ 万能：匹配查询习惯

- **裁剪**：WHERE 命中分区键 → 优化器自动裁剪。
- **显式指定**：SQL 直接写分区。
- **核心原则（必背）**：**分区键 + 边界必须围绕常见 WHERE 设计**。否则形同虚设，维护成本 ↑。

J. DROP 中间分区会发生什么？

- **RANGE**：逻辑仍连续。后续分区 **自动覆盖**。新数据正常插入（落入新分区）。

K. 分区调整规则（易考）

- **RANGE**：✓ 只能在末尾追加。× 不能在中间插。中间调整 → `REORGANIZE`（重写数据）。
- **LIST**：可加新值/新分区。不能重复。已有值重分 → `REORGANIZE`。
- **HASH/KEY**：合并易，拆分难。增分区不支持或代价极高。

L. 分区交换（Partition Exchange）

- **是什么**：分区表某分区 ↔ 普通表 **直接交换**。
- **为什么快**：只换元数据指针。无逐行 INSERT/校验。瞬时。
- **限制**：普通表非分区、结构完全匹配。

- **校验**：默认查范围。强关校验 → 数据落错分区 → 维护灾难。

A. 什么是 NoSQL？(概念澄清)

- **NoSQL ≠ No SQL**。更准确理解：**NoSQL = Not Only SQL**。
- **含义**：不是要“取代”关系型数据库，而是在 SQL 不擅长的场景下，提供另一套存储模型。

B. 为什么需要 MongoDB？(动机)

- **核心原因**：数据不再全是结构化的。系统变大 → **半结构化 / 非结构化**，Schema 经常变化。
- **典型**：日志、评论/书评、商品参数、用户画像、配置数据。
- **对比**：关系型强 Schema + 固定列；MongoDB 支持 **Schema-free**，更灵活。

C. 什么是 BigTable？(思想来源)

- **Big**：能存海量数据，能水平扩展。
- **Table**：看起来像表，但不是传统关系表。
- **影响**：HBase, MongoDB, Cassandra, DynamoDB。

D. 关系型数据库分布式切分之痛

- **灾难点**：多对多关系（如 Student ↔ Course）。
- **问题**：切分后关联查询必然 → **跨机器 JOIN** → 网络跳转频繁 → 性能灾难。
- **NoSQL 思路**：减少跨表关联；尽量把相关数据放在一起；用更容易水平切分的数据模型。

E. MongoDB 的核心特性（必背）

- **面向文档 (Document-Oriented)**：最小单位 Document。
- **索引丰富**：普通/地理/2D。
- **高可靠**：Replica Set（副本集）。
- **可扩展**：Sharding（分片），自动 Balancing。
- **查询友好**：语法近 SQL。
- **更新高效**：支持 **in-place update**（就地更新）。
- **其他**：MapReduce, GridFS（大文件），云服务。

F. 数据模型（对照）

- MongoDB ≈ RDBMS:
- **Document** ≈ Row; **Collection** ≈ Table; **Database** ≈ Database。
- **补充**：Document 本质是 **BSON/JSON**。字段可不同、可嵌套、可数组。

G. 关系约束谁来管？

- **原则**：不强制维护引用，无外键约束，无级联检查。
- **结论**：关系一致性由 **应用层自己保证**。
- **取舍**：✓ 性能/扩展性；× 强一致性约束。

H. 为什么还要分 Collection？

- **能全丢一个吗？** 不应该。
- **原因**：语义清晰；**索引成本**（混在一起索引巨大难维护）。
- **原则**：按业务语义拆，而不是按“技术偷懒”。
- **注**：`blog.post` 只是命名习惯，Mongo 无层级结构。

I. Mongo 就是存图片的吗？

- × **误解**。真正擅长：**Schema-free 的文档数据**（同 Collection 结构不一致/随业务演进）。

J. Ebookstore 能全用 Mongo 吗？

- ✓ **可以**。NoSQL = Not Only SQL。
- 若让所有 Document key 相同 → 可像关系表一样存结构化数据。技术取决于需求。

K. 2D / 地理索引

- **数据**：`[x, y]` 或 `[long, lat]`。
- **查询**：半径 R 内的点；矩形范围。
- **场景**：附近的人/店/LBS。

L. 手机参数案例（为什么适合？）

- **问题**：不同品牌参数不同，未来不可预判。
- **关系表**：不断加列，大量 NULL，维护难。
- **MongoDB**：`parameters: [ {name:"screen",...}, {name:"price",...} ]`。
- **结论**：字段可变、结构不固定 → MongoDB 强项。

M. 分布式核心组件（必考）

1. **Shard（分片）**：水平扩展单元。解决单机放不下/扛不住。
2. **Shard Key（分片键）[最重要]**：决定数据切分与热点。选不好再多机器也慢。
3. **Chunk**：Shard 内部数据块（按 Key 范围）。太大自动 split。
4. **Config Server**：存元数据（哪个 Chunk 在哪个 Shard）。
5. **Mongos（路由）**：客户端入口。查 Config → 转发请求。
6. **Balancer（自动均衡）**：
   - 监控 Chunk 分布 → 不均衡自动迁移。
   - **局限**：按**数据量**均衡，不按**访问量**。热点 Chunk 仍可能压垮 Shard。
   - **解法**：选好 Shard Key（必要时 Hash 打散）。
7. **Replica Set（副本集）**：每个 Shard 是一个副本集（Primary 写/Secondary 读）。保高可用。

N. 分布式一句话总结

- **Shard**：水平扩展；**Chunk**：数据切片；**Shard Key**：成败关键；
- **Balancer**：自动均衡；**Mongos**：路由；**Replica Set**：可靠性。

A. 为什么要用图数据库？(动机)

关系型数据库的根本问题

- 关系是表级抽象，不是实例级直连。
- 查“关系链”必须：多表 JOIN + 多次索引查找。
- **难表达**：“走 1~N 跳能到哪些对象?”、“两人间最短路径?”

典型例子：“Alice 买过什么？”

- **RDBMS**: User → Order → OrderItem → Product（多表 JOIN，成本随深度上升）。
- **Graph**: User/Order/Product 都是**节点**，“购买”是**边**。
- **做法**：找到 Alice → 沿边直接走。多跳查询天然高效。

核心优势

- 实体 = 节点；关系 = 边（**一等公民**）。
- **擅长**：多跳关系、路径查询、关系遍历。

B. Graph DB vs MongoDB（关键区分）

- **MongoDB**：擅长文档，不保证外键，**不适合**多跳。
- **Graph DB**：天然建模关系，多跳成本低，适合“关系密集型”。
- **结论**：MongoDB 解决“**结构问题**”，图数据库解决“**关系问题**”。

C. 异构实体的统一建模

- **允许**：User, App, VM, Server, Rack → **全部作为节点**。
- **关系**：统一用**边**表示。
- **好处**：统一视角，不需要多套表结构。
- **场景**：IT 资源管理、依赖分析、权限/拓扑。

D. 建模：属性还是节点？(必考)

核心原则：未来要不要“**查 / 连 / 走**”的东西，就该建成节点。

- 实体（things）→ 节点；联系 → 边。
- 但某个信息是属性还是节点，取决于**未来需求**。

举例（城市）

- 只用于展示 → **属性**。
- 需查“同城用户 / 距离 / 迁移路径” → **节点**。
- **结论**：不是语法问题，是**建模前瞻问题**。

E. 图太大怎么办？(分布式图)

- **问题**：单机存不下。
- **方案**：**子图 (Subgraph) 切分**。分布到不同机器。
- **关键取舍**：不能破坏关系语义 → 需要**少量冗余节点**（共享节点在各子图各存一份）。
- **好处**：查询并行，支持副本/读写分离/负载均衡。

F. 图计算 & 图神经网络（GNN）

常见任务：节点分类（关键人物），边预测（好友），子图分类（洗钱模式）。**为什么要 GNN?**：规则太复杂 → 自动学习模式。**两大思路**：

1. **谱方法**：把图当信号，分析相似性。
2. **消息传播（主流）**：节点从邻居聚合特征，逐层捕捉多跳。

注意：图数据库（偏查询）≠ 图神经网络（偏学习）。

G. 两个“致命缺陷”（考试重点）

1）标准化差

- 各家语言不统一（如 Neo4j Cypher）。跨产品迁移难。
- 对比：RDBMS 有 SQL。新标准 **GQL (2024)** 刚出现，生态未熟。

2）底层存储不是真正“图优先”

- 逻辑上是图，底层仍多是：记录/数组/KV。
- 对复杂算法（BFS/DFS/社区发现）支持有限。
- **结论**：图数据库更适合 **关系遍历/路径查询**，**不等同于**高性能图计算引擎。

Part A. Log Database (LSM-Tree / RocksDB)

1. LSM-Tree 解决什么问题？

- **核心目标**：适应冷热数据分布，优化**高写入**。
- **机制**：新数据（热）在 MemTable（内存查询快）→ 老数据下沉 Compaction（底层冷，访问慢可接受）。
- **相比传统 SQL**：更适合 **持续写入 + 时序/日志**。

2. 为什么 MemTable 要“多块”？

- **防止写入阻塞**。
- **单块**：写满 → 刷盘 → 阻塞。
- **多块轮换**：当前写满 → 变成 **Immutable**（后台刷盘）→ 切换新 MemTable 继续写。
- **效果**：可能同时存在多个 Immutable，保证吞吐不降。

3. LSM-Tree 的优点

- **写性能高**：顺序追加写（顺序 IO）。更新/删除 = 新版本或**墓碑 (tombstone)**。
- **空间放大低**：无重型索引，冷数据压缩。
- **热数据读快**：查找路径短（MemTable/高层）。
- **场景**：日志、时序、实时写入。

4. LSM-Tree 的缺点

- **写放大**：多层 Compaction 级联。
- **读放大**：冷数据/不存在数据需查多层。
- **写阻塞**：刷盘/Compaction 抢资源 → 延迟抖动。

5. SSTable 如何节省空间？

- **Block 内前缀压缩 (Prefix Compression)**。
- Key 按共享前缀分组。每组：共享前缀存一次 + (共享长/非共享长/value 长)。
- **Block 底部**：Group Offset + CRC。
- **原理**：空间换结构复杂度。

6. RocksDB 适合分布式吗？

- **结论**：**不适合直接做分布式数据库**。
- **擅长**：单机、嵌入式、高写入。
- **不擅长**：跨机一致性、分布式 Compaction（Compaction 是本地行为）。
- **定位**：单机存储引擎。

7. 典型矛盾（必考）

- **OLTP 强**：小写入/高并发 → LSM 适合。
- **OLAP 弱**：大扫描/聚合。行存 → 逐条解析 → 分析差。

8. 一行很宽怎么办？(Column Family)

- 把宽记录按字段分组 → 不同列族存不同 SSTable。
- **目的**：避单行过大。
- **注**：真正擅长宽表的是 Cassandra，不是 RocksDB。

9. 解决写阻塞的手段

- 前置 Kafka/缓存做缓冲。后端 Flink 慢写。
- Master 调度：压力轻的节点优先。

10. 解决读放大的思路

- 增加 **列存**处理（适合 HTAP）。

Part B. Vector Database (向量数据库)

11. 为什么不能用普通 List？

- 高维计算代价极高。向量库用近似/索引技术加速。
- **技术**：
  - **随机投影**：高维 → 低维。
  - **乘积量化 (PQ)**：切块聚类，用 ID 代替原值。
  - **LSH**：局部敏感哈希（相近哈到同桶）。
  - **HNSW**：分层小世界图（自顶向下逼近）。
  - **核心思想**：**牺牲精确度，换速度**。

12. 过滤策略

- **后过滤**：先向量搜 → 再条件筛。✓ 召回高；× 性能差。
- **先过滤**：先条件筛 → 再向量搜。✓ 性能好；× 可能漏结果。

13. Pinecone 云化架构

- 分布式分片（Shard），一主多从。
- **流程**：路由 → 分片 → 向量索引。适合云原生。

一、为什么需要时序数据库？

- **RDBMS 能存但不擅长**：查询按时间范围；写入是顺序追加。时间天然有序 → 传统索引收益有限。
- **真正的瓶颈**：
  - 高并发、高吞吐写入。
  - 压缩率与存储效率。
  - 时间范围聚合 / **下采样 (downsampling)**。
  - 数据保留与过期删除（TTL）。
- **TSDB**：为“**时间维度优先**”的读写模型而生。

二、InfluxDB 数据模型

- **timestamp**：时间戳（主轴）。
- **measurement**：指标类别（“是什么”）。
- **tags**：可索引标签（过滤用）。
- **fields**：具体数值（不建索引）。

三、Tag vs Field（必考）

- **Tag**：参与索引 → 过滤快 → 适合查询条件。
- **Field**：不参与索引 → 过滤慢 → 适合存数值。
- **原则**：经常出现在 WHERE → Tag；只是记录数值 → Field。

四、InfluxDB 基本概念

- **Bucket**：顶层容器（类似表），固定 schema。
- **Point**：某时间点的一条记录。
- **Series**：相同 tag 组合形成的一条时间序列（多 point）。

五、写入模型：为什么快？

- 按时间 **顺序追加**（类 LSM/日志）。
- 几乎无主键冲突，无需频繁更新。
- **公式**：**顺序写 + 少冲突 = 高吞吐**。

六、数据增长：保留策略（TTL）

- 数据天然无限增长。Bucket 设置 **Retention / TTL**。
- **作用**：超期自动删除 → 控制规模，只留业务时间窗口。

七、三大难点（重点）

1. 并发写入 vs 时间有序

- **服务端时间**：顺序好，但非真实采集时间。
- **设备时间**：准确，但会产生 **乱序写入**。

2. 几乎不支持更新/删除

- 模型假设只追加（更新破坏顺序写）。
- **做法**：限制不支持，依赖 TTL 自动淘汰。

3. 一致性策略

- 不追求强事务。允许查时漏掉最新少量数据 → **最终一致性**。

八、数据库体系对比总结（总复习）

1）KV / 内存（Redis）

- **优**：Key 查找极快。**短**：复杂查询难，关系/索引需应用维护。**结论**：适合缓存，**不适合主库**。

2）MapReduce / Hadoop

- **定位**：分布式批处理。**短**：重量级，大量磁盘 IO，编程复杂。**结论**：超大规模 **离线处理**。

3）图数据库（Graph）

- **优**：关系建模直观，减少 JOIN。**短**：标准化晚，底层存储可能不适高性能算法。**结论**：**强建模，弱通用性能**。

4）MongoDB（文档）

- **优**：Schema 灵活，易扩展，开发快。**短**：事务弱，强一致性复杂。**结论**：**“够用型”系统**。

5）向量数据库（Vector）

- **优**：相似度检索（ANN）极强。**短**：不擅长事务/关系/复杂约束。**结论**：专业工具，只解相似检索。

6）宽表体系（HBase/BigTable）

- **解**：字段不统一、稀疏数据。**特**：列族模型，行不要求同构。**差**：语义（不存在 vs NULL），存储扩展方式。

7）为什么 RDBMS 难被取代？

- 事务与一致性强。SQL 标准成熟生态统一。
- 复杂业务逻辑表达力强（JOIN / 约束 / 聚合）。

Intro：为什么需要数据湖？

- **背景**：系统既有多种数据库，又有海量文件/对象 → “异构数据”。
- **问题**：如何统一存、管、处理？→ 引出 Data Lake。

1.2 数据湖是什么？(必考)

定义：面向多源异构数据的“**原始数据仓库**”。**核心特征**：

1. **Raw / Native format**：原始格式存储。不强制转结构，直接落盘。
2. **动机**：不知道数据何时用、怎么用 → **先快速摄入 (Ingest)**，用时再加工。
3. **核心权衡（必考）**：
   - **优**：写入/摄入快、吞吐高、可线性扩展。
   - **代价**：读取/分析时慢（解析/清洗推迟到用时）。

1.2 补：ETL (Extract-Transform-Load)

- **E**：抽取（MySQL/Log/JSON...）。
- **C**：清洗（补值/删脏数据）。
- **T**：转换（统一口径）。
- **L**：加载（入仓）。
- **Data Lake 关键**：**不提前把 ETL 做完**。

1.3 能存哪些数据？(来者不拒)

- **结构化**：表。
- **半结构化**：CSV / JSON / XML (Schema 弱)。
- **非结构化**：Log / TXT。
- **二进制**：图片 / 视频。
- **类比**：“巨大仓库/湖泊”，先倒进去，未来拆包。

1.4 落盘：对象存储 + 分布式

- 基于 **Object Storage** (S3 / HDFS / 自建)。
- 一切当 Object 存。关键能力：分布式扩展、容错、高吞吐。

1.5 数据处理三件套

1. **流 (Streaming)**：事件驱动。来一条处理一条。
2. **批 (Batch)**：定期/凑批。攒一批再算。
3. **流批一体 (Unified)**：微批 (Micro-batch)。窗口缩到秒级 → 宏观像流（近实时）。

1.6 Data Lake vs. Data Warehouse（必考）

数据仓库 (DW)

- **机制**：先处理、再分析。重 ETL。
- **特点**：强结构化（宽表/Cube），面向 BI。
- **优**：查询快。

数据湖 (DL)

- **机制**：**先存再说**、用时再加工。
- **特点**：Raw 格式，面向探索分析/数据科学。
- **优**：摄入快、灵活。代价：查询可能慢。

一句话：数仓 = 前置加工换查询快；数据湖 = 先存换灵活与摄入快。

1.7 典型平台分层架构

- **存储**：HDFS / S3。

- **资源调度**: YARN (分配任务到机器)。
- **计算引擎**:
  - **MapReduce**: 落盘多 → IO 重、慢。
  - **Spark**: **内存计算** → 加速。
- **接口**: Hive / SQL (统一查询语义)。
- **协调**: ZooKeeper。
- **治理**: **Metadata / Data Catalog** (知道湖里有什么)。

1.8 演化: Lakehouse (湖仓一体)

- **趋势**: 湖本身具备 "仓" 的能力 (事务/结构化视图)。
- **能力**: 既支持原始读取，也支持结构化分析。常配合流批一体。

1.9 三个代表方案

Delta Lake

- 底层 S3/HDFS。支持流/批。
- **经典分层**:
  - **Bronze**: 原始摄入 (先存)。
  - **Silver**: 清洗/规范化。
  - **Gold**: 聚合/指标表 (精品)。
- 分布式才体现价值。

Apache Hudi: 强调 "事件流摄入" (增量 ETL)。**Apache Iceberg**: 表格式/管理层方案。大厂常用。

优势总结

- **灵活**: 什么都能存。
- **可扩展**: 存不下加机器。
- **成本低**: 廉价存储 + 软件容错。

1. 应用集群核心难点: 会话 (Session)

- **目标**: 后台服务尽量做成 **无状态 (Stateless)**。
- **好处**: 任意分发，宕机可重试。
- **现实**: Session 本身就是状态，很难完全无状态。

2. Session 解决方案 (重点对比)

2.1 状态放在客户端

- **方式**: LocalStorage / Cookie。
- **问题**: 易篡改 → 安全风险；需签名/加密。
- **结论**: × **一般不推荐**。

2.2 集群内同步/迁移

- **方式**: 广播/拷贝 Session。
- **问题**: 广播成本高，版本冲突，GC 开销大。
- **结论**: 可行但复杂，**不优雅**。

2.3 状态外部化 (最常用)

- **方式**: Cookie 存 ID → 数据放 **Redis**。
- **优点**: 任意节点可处理，无须同步。
- **风险**: Redis 挂 → 全挂。需 Redis 高可用 (3 副本)。
- **结论**: ✓ **工程中最推荐**。

2.4 Sticky Session (固定路由)

- **方式**: IP Hash。
- **缺**: 负载不均 (大量用户共享出口 IP)。
- **结论**: 简单但有明显缺陷。

3. 核心问题: 幂等性 (Idempotency)

- **场景**: A 崩 → LB 重试 B → 无法判断 A 是否已完成。
- **解决 (应用层)**: 幂等键 (Key)、去重逻辑、状态机。
- **解决 (数据层)**: 关键状态写 RDBMS (事务 + WAL) 保证 "只一次"。

4. Nginx 负载均衡策略

- **Round Robin**: 轮询 (默认)。
- **IP Hash**: 固定节点。
- **Least Connection**: 最小连接数优先。

5. 反向代理 (Reverse Proxy)

- **配置**: `upstream` (定义后端), `proxy_pass` (转发)。
- **区别**: 正向代理 (代 Client 访外); 反向代理 (代外部访内部)。
- **建议**: 后端只暴露内网，防绕过 Gateway。

6. App 集群 vs DB 集群 (必考)

应用集群 (应用层)

- **难点**: **瞬时状态** (Session/购物车)。
- **策略**: 无状态化、Redis 外部化。

数据库集群 (数据层)

- **难点**: **持久化状态一致性**。
- **依赖**: 复制、日志 (WAL)、选主、共识。

7. MySQL 集群 (主写从读)

- **流程**: Primary `createCluster` → `addInstance` 加 Secondary (Clone)。
- **Docker**: 必须在同一 `network`，否则无法通信。

8-9. 故障与同步策略 (必考)

主从故障

- Secondary 崩 → 补新。Primary 崩 → 选举/Failover。

同步策略

- **异步复制**: 写完即返。快，但最终一致 (从库落后)。
- **同步复制**: 等从库写完。强一致，但慢、失败率高。

10-11. 权重与多主

- **权重**: Nginx weight (分流比例) vs MySQL memberWeight (选主优先级)。
- **多主 (Multi-Primary)**: 吞吐高，但**写冲突**复杂，工程谨慎。

12. Nginx Request Object

- TCP 连接 ≠ 单请求 (Keep-alive 复用)。
- **Request Object**: 逻辑层唯一标识，精确匹配 Response。

14. Nginx 出错的三个点 (非常重要)

1. **建连失败**: 立刻可见，易处理。
2. **请求转发时失败**: TCP 已建，写 request 失败 → 快速重试。
3. **请求已发出，等待响应时后端挂 (最麻烦)**:
   - 无法分清 "慢" 还是 "死"。
   - 只能靠 **超时机制 (Timeout)**。

15. 为什么 Nginx 不是瓶颈？

- 工作极轻: 映射 + I/O 转发 + 少量容错。
- **不做**业务计算 / 复杂逻辑。主要是 I/O + 状态管理。

一、Cloud Computing (云计算)

1. 云计算是什么？核心思想云计算不是 "某几台很大的服务器"，而是一整套 **资源管理与抽象体系**。

1) 资源抽象与虚拟化

- **用户**: 不关心机器在哪、型号。只看到: 虚拟机/容器/存储/网络/服务接口。
- **本质**: 把物理资源抽象成可分配的逻辑资源。

2) 统一调度 (像操作系统)

- **云平台 ≈ "超大规模操作系统"**。
- **负责**: 把任务/容器/VM 调度到合适机器; 资源隔离; 自动扩缩容。
- **用户**: 不再 "买机器"，而是 "申请资源"。

3) 屏蔽复杂性 (不透明)

- **云平台负责**: 容错、扩容、负载均衡、监控、备份。
- **用户关注**: 业务逻辑、接口与数据。

总结: 云计算 = 把复杂的基础设施问题，全部**平台化、自动化**。

二、GraphQL

2.1 SOAP → REST: 解决与遗留

- **SOAP 问题**: XML+Envelope 极其重量级; 接口强绑定，演化难; 序列化成本高。
- **REST 解决**: HTTP+JSON 轻量; URL 语义清晰; 前后端解耦。
- **REST 新问题**:
  - **Over-fetching**: 返回过量数据 (只要 name 却返整个对象)。
  - **资源组合难**: 跨实体/层级数据笨重。
  - **N+1 问题**。

2.2 REST 的 N+1 问题 (必考)

- **典型模式**: 1 次请求列表 → N 次逐个请求详情。
- **例**: `/authors/{id}/books` (列表) → `/books/{bookId}` × N。
- **问题**: 网络往返多，延迟高，性能差。
- **根因**: REST 以 "资源 +URL" 为核心，关联只能 "逐个追链接"。

2.3 GraphQL 能解决什么？ 核心思想: 客户端声明 "我要什么结构和字段"，服务端按声明一**次性返回**。

- **1) 避免 Over-fetching**: 前端要 A → 后端只返 A。
- **2) 缓解 N+1**: 一次请求拿到列表 + 关联对象 + 所需字段。
- **3) 减少后端 DTO / 组合接口爆炸**: 只有一个 `/graphql` 入口。返回结构由 query 决定。

2.4 GraphQL 的问题 并不是银弹:

- 返回仍是 **JSON 文本**。
- **代价**: 体积大 (字段名/字符串)，解析慢。
- **对机器通信不友好**: 微服务/IoT/高吞吐场景。
- **机器更想要**: 二进制、强结构、高性能。

三、gRPC

3.1 gRPC 为什么出现？弥补 REST / GraphQL 在**机器通信**上的不足。

- JSON 太慢太大 → **二进制 (Protobuf)**。
- 接口不统一 → **强契约 (.proto)**。
- 跨语言麻烦 → **自动生成代码**。
- 一问一答不够 → **原生流式通信**。

3.2 gRPC 的核心机制 1) Protobuf (.proto) 定义契约

- `message`: 数据结构; `service`: 远程方法。
- **字段编号 (=1)**: 用于高效二进制编码 (给机器看的)。

2) 二进制序列化

- 体积小，解析快。适合微服务/高吞吐/IoT。

3) 自动生成 Stub

- 从.proto 自动生成 Java/Go/Python/C++ 代码。
- 开发者只写业务，不关心网络/序列化细节。

4) RPC + Streaming 模型 (重点)

- **Unary**: 一次请求，一次响应。
- **Server Streaming**: 客请求一次，服持续推送。
- **Client Streaming**: 客持续发送，服一次返回。
- **Bidirectional Streaming**: 双向持续流式通信。

1. 容器的本质: Sandbox (沙盒)

- 容器不是 "新机器"，本质仍是 ** 宿主机上的一组 Linux task**。
- 只是把 Task ** 隔离 + 限额 + 换根目录 ** → "看起来像独占"。
- **Linux 视角 **: 进程/线程 ≈ Task。容器 = 被隔离的 Task 组。

2. 容器隔离三件套 (必背)

容器 "像机器" 的效果来自 3 个机制:

2.1 Namespace: 控制 "能看见什么" (隔离视图)

- 隔离资源视图，让容器以为 "世界只有我"。
- **PID**: 自己的进程树; **NET**: 自己的 IP/端口; **MOUNT**: 自己的挂载。
- **关键词**: 可见性隔离 / 独立视角。

2.2 cgroups: 控制 "能用多少" (资源配额)

- 防止抢光宿主机资源。
- **Memory**: 内存上限; **CPU Quota**: CPU 时间片。
- **关键词**: 配额 / 限流 / 资源隔离。

2.3 rootfs / chroot: 控制 "根目录是什么"

- 容器内路径从 / 开始。需把宿主机某目录映射为容器内的 /。
- **关键词**: 换根 / 文件系统映射。

2.4 一句话总结

- Namespace: 能看见什么。
- cgroups: 能用多少。
- rootfs: 看到的根目录是什么。
- **本质**: 仍在同一个内核上跑 Task。

3. 镜像 (Image) vs 容器 (Container)

- ** 镜像 **: 应用运行环境打包 (OS 基础 + 运行时 + 依赖 + 代码)。
- ** 容器 **: 镜像的一个 ** 运行实例 **。
- 'docker run': Image 作 rootfs → 配 NS/cgroups → 跑 Task。

4. 镜像分层 (Layer): 蛋糕模型

- ** 为什么分层?** 复用 OS/依赖层 → 减少下载，省磁盘。
- **Copy-on-Write (写时复制)**: 启动不复制文件。只在写入时产生 ** 可写层 (变更层)** → 启动快。

5. 容器 vs 虚拟机 (VM)

- **VM**: 虚拟硬件 + 完整 OS → 重、慢。
- **Container**: 共享宿主机内核 + 仅带文件系统 → 快、密度高。

6. 基本工作流

- Dockerfile (描述) → 'build' → Image → 'run' → Container。
- Registry (仓库): 'push' / 'pull'。

7. 关键坑: 默认不持久化 (必考)

- 写入发生在容器 ** 可写层 **。
- ** 删容器 = 可写层没了 ** → 数据丢失。
- Image 不变。

8. 持久化 1: Volume (卷)

- ** 是什么 **: 类似 "外挂硬盘"。不属镜像。容器删了 Volume 仍在。
- ** 存哪 **: 宿主机特定目录 (Docker 管理)。
- ** 场景 **: ** 生产/长期数据 ** (DB 数据)。便于迁移管理。

9. 开发痛点: 改代码要重 build

- 代码打进 Image → 改一行须重 build + run → 效率低。

10. 持久化 2: Bind Mount (挂载)

- ** 是什么 **: 把宿主机 ** 任意目录 ** 映射进容器。** 实时映射 **。
- ** 对比 (必背)**:
  - **Volume**: Docker 管理的 "外挂盘"。
  - **Bind Mount**: 指定宿主机路径 "直接射进去"。
- ** 场景 **: ** 开发调试 / 热更新 ** (本地改代码 → 容器生效)。

11. 多容器网络 (Network)

- ** 为什么 **: 同一 Network 才能互通 + 用容器名当 DNS 解析 (如 'db_host=mysql')。
- ** 流程 **: 建网 → 容器加入 → 容器名互访。

12. Docker Compose (编排)

- ** 是什么 **: 'docker-compose.yml' 描述一组服务。
- ** 功能 **: 一键 'up' (启动整套) / 'down'。
- ** 默认 **: 会建一个 Network → 容器名天然可用。
- ** 收益 **: 编排、复现、可移植。

13. Compose → K8s

- **Compose**: 单机多容器。
- **Kubernetes (K8s)**: 多机集群。
- **K8s 目标 **: 在容器机制 (NS/cgroups) 之上做 ** 调度 + 编排 + 自愈 + 伸缩 ** (在哪跑/跑几个/崩了咋办)。

14. 为什么开发喜欢 Docker?

- 省本地环境配置 (多版本 Node/Python)。
- 拉镜像即用 (MySQL/Redis 快速起)。
- **Bind Mount** 让改代码立刻生效。
- ** 代价 **: 多一层运行时性能损耗。
- ** 总体 **: **一致性、可移植性、效率更高**。

1. Hadoop 核心组件

hadoop-common: 基础组件库。
HDFS: 分布式文件系统 (存储)。
MapReduce: 分布式计算框架 (Map/Reduce 逻辑)。
YARN: 资源管理与调度 (集群操作系统)。

2. HDFS Node 区别

NameNode: 保存 **元数据** (Metadata)。

- 文件切成多少块 (Block)。
- 每个块分布在哪些 DataNode 上。

DataNode: 真正存储文件块 (Block) 的节点。

3. MapReduce 机制

角色:

- **Master**: 解析程序、切分任务、调度 Worker。
- **Worker**: 执行 Map/Reduce 计算。

Split (输入切分):

- **大文件**: 按固定大小 (如 64MB) 切块 → 并行处理。
- **小文件**: **先合并**再处理 (避免 Task 过多)。

容错: Worker 崩溃/过慢 → 重试/重调度; 多次失败 → Job 失败。
Shuffle (洗牌): Map 输出 KV → 按 Key 分区 → 同 Key 送同一 Reducer。
Reduce: 拉取数据 → 聚合/统计 → 输出。

4. Combiner (可选优化)

定义: Mapper 本地的 "小型 Reduce"，减少中间数据传输。
机制: 本地合并 $(K,1) \times 3 \to (K,3)$。Reducer 再汇总。
条件: 仅适用于 **可交换、可结合**操作 (Sum, Max)。并不是必须组件。

5. 配置策略: Split/Map/IO

Split vs Map: 1 Split 通常对应 1 Map，但物理 Map 数有限 (如 1000 Split 只有 100 Slot)，需分批执行 (队列)。
小文件处理: 必须聚合 (Combine) 成大 Split，让 1 Map 处理多个小文件 → 降低调度开销。
瓶颈: Map/Reduce 间不是内存传递，而是 **重度磁盘 I/O**。

6. 分区 (Partition) 与 Map-only

分区策略:

- **Hash**: $Hash(key)\%R$ (常用)。
- **Range**: 按 Key 范围切分。

Map-only: 无需聚合 (如格式转换/清洗)。不设 Reducer，无 Shuffle。

7. JobTracker (MRv1 核心)

职责: 跟踪 Job 状态 (Task 是否完成/失败); 监控 TaskTracker。
心跳机制 (Heartbeat):

- **独立线程定时 (10s)**: 与 Task 主逻辑解耦。缺: 任务死锁但心跳正常 (假健康)。
- **嵌入执行流程**: 真实反映推进。缺: 执行时长不同导致心跳间隔不稳。

缺陷: 单点故障 (SPOF); 既管资源又管任务 → 性能瓶颈。

8. 中间结果 Reduce 流程

中间文件: Map 输出写本地磁盘 (非内存); KV 使用 Writable 序列化。
流程: Shuffle → Sort (可选 Secondary Sort) → Reduce。
Reduce 数量: ≈ 容器数 × 系数 (太多切换重)。

9. 性能优化

压缩: 压缩 Map 输出 → 减磁盘/网络 I/O。
Spill (溢写): 内存 Buffer 满 → 写盘; 多次 Spill → Merge。
JVM/GC: 内存不足/GC 频繁 → 任务抖动/超时。

10. 为什么 MR 重度依赖磁盘 I/O?

1. **中间结果落盘**: Buffer 满即 Spill; 多次 Spill 需 Read-Merge-Write。

2. **Shuffle 开销**: 写本地磁盘 → 网络传输 → Reducer 读盘/排序/写盘 (双重 I/O)。
3. **输出落盘**: 结果写回 HDFS。
4. **容错成本**: 失败触发重算 → 重复 I/O。

优化思路: 让 Spill 尽早切分推送 Reducer; 崩溃时仅重算缺失部分 (复杂)。

11. YARN: 资源/调度解耦

MRv1 痛点: JobTracker 压力大 (资源 + 任务一把抓); 单点故障。

YARN 架构 (拆分职责):
- **ResourceManager (RM)**: 全局资源管理 (只管分资源)。
- **NodeManager (NM)**: 单机资源汇报。
- **ApplicationMaster (AM)**: 单个作业的任务调度、监控、重试。

优势: 解耦; 隔离 (AM 挂不影响其他作业); RM 变轻 (稳定)。

23.1 Spark: 动机与核心

MR 痛点:
- **重度磁盘 I/O**: 中间结果反复落盘 → 慢。
- **纯批处理**: 延迟高, 不适合实时。

Spark 优势:
- **内存计算**: 中间结果优先放内存 → 速度显著提升。
- **流批一体**: 统一引擎。批 (离线大规模) + 流 (微批实时)。

快的原因: 计算移至内存 + 减少 Shuffle (窄依赖流水线)。

RDD (弹性分布式数据集)

定义: 分布在多节点上的 "同一份数据集合"。

核心:
- **Partition (分区)**: 数据按区分布, 并行计算。
- **构建**: `textFile` (外部) 或 `parallelize` (内存集合)。

三要素: 弹性 (容错重算); 分布式 (物理多机); 数据集 (逻辑一份)。

架构与惰性执行

结构:
- **Driver**: 生成任务、切分 Stage、调度。
- **Executor**: Worker 进程, 执行分区计算。

惰性执行 (Lazy Evaluation):
- **原则**: 遇到 Action 才触发真正计算 (回推)。
- **好处**: 省内存 (中间结果不提前算); 省 CPU; 允许 Pipeline 优化 (合并算子)。

Transformation vs Action (必考)

Transformation (变换):
- 输出仍是 **RDD**。描述 "怎么变"。
- 例: `map, filter, flatMap, union`。

Action (动作):
- 输出是 **值/结果**。触发执行。
- 例: `reduce, count, collect` (拉回 Driver)。

依赖: 窄 vs 宽 (Stage 划分依据)

1. 窄依赖 (Narrow):
- 父分区 → 子分区 (1 对 1)。
- **无需 Shuffle** (本地计算)。
- 例: `map, filter, union`。
- 特点: 快、流水线执行、恢复快。

2. 宽依赖 (Wide):
- 父分区 → 多个子分区 (1 对多)。
- **必须 Shuffle** (重分发)。
- 例: `groupByKey, reduceByKey`。
- 特点: 慢 (IO/网络)、Stage 切分点。

DAG 与 Stage

DAG: 算子组成的有向无环图。

Stage 切分点 (必背):
1. **宽依赖 / Shuffle**: 遇到 Shuffle 必须断开。
2. **Persist / Cache**: 缓存点作为边界。

作用: Stage 内全窄依赖 → 流水线 (Pipeline) 高速执行; 失败从上个 Stage (或 Cache) 重算。

Shuffle 分区 (Join 优化)

Shuffle: 按 Key 重新分区分发。代价: 磁盘 + 网络。

Join 优化: 若两表按同 Key (如 UserID) **提前分区并 Cache**:
- Join 变为 "分区对分区" (本地化)。
- 避免 Shuffle → 极大提速。

Cache / Persist

场景: 某 RDD 后续被反复使用 → `cache()`。

机制: 内存不足可淘汰 (LRU) 或落盘。

级别: 仅内存; 内存 + 磁盘; 序列化 (省内存耗 CPU)。

Spark Streaming: 微批

原理: 将流数据按时间片切成 **Micro-batch**。

本质: **用批模拟流**。

权衡: 时间片小 ≈ 实时; 时间片大 ≈ 批处理。

23.2 Storm: 纯流式

定位: 专门做流处理, 低延迟 (来一条处理一条)。

Topology (拓扑):
- **Spout**: 水龙头/数据源。
- **Bolt**: 处理节点 (转换/统计)。

并行: 多实例并行。传输支持 Shuffle (随机) 或 Fields (按字段 Hash)。

1. HDFS 定义与适用性

定义: 面向大文件的分布式文件系统。目标: 低成本、强可靠存储海量数据。

不适合场景 (必考):
1. **低延迟随机访问**: 数据分布多节点, 读需跨网络/磁盘 + 查 NN 元数据 → 慢。
2. **海量小文件**: 文件切 Block, 元数据由 NN 维护。小文件太多 → **元数据 (内存) 爆炸**。
3. **频繁修改/多写并发**: 不支持任意位置修改 (Append-only); 同一时刻 **单 Writer**。

2. 主从架构 (Architecture)

NameNode (NN): 管理 **元数据** (目录树、权限、Block 与 DN 映射)。

DataNode (DN): 存储实际数据块 (Block)。

Block 副本:
- 文件切分 Block (默认 **128MB**)。
- 默认 **3 副本** (容错)。

3. 读写机制 (核心)

读 (Read): 就近原则。
- Client 问 NN 位置 → Client 直接连最近 DN 读取。
- 数据流 **不经过 NameNode** (避免瓶颈)。

写 (Write): Pipeline (流水线)。
- 数据流: Client → DN1 → DN2 → DN3。
- **ACK**: 所有副本写完确认才算成功。
- 权衡: 副本多 → 写得慢但越安全。
- 限制: 单文件单写者。

4. DataNode 职责 容错

定位: 只认识 Block, 不认识文件概念。

汇报机制:
- **Heartbeat**: 报活。
- **Block Report**: 汇报持有块信息。

故障恢复: DN 挂 → NN 标记死亡 → 挑选新节点复制副本 (维持副本数)。

均衡: 系统自动做数据均衡 (Balancer)。

5. 启动: 安全模式 (SafeMode)

流程: NN 启动 → 进入 SafeMode (只读) → 接收 DN 的 Block Report → 副本率达标 → 退出 SafeMode 对外服务。

6. NameNode 持久化 (元数据)

位置: 必须在本地磁盘 (防止内存掉电丢数据)。

EditLog (编辑日志):
- 记录 **修改操作** (创建/删除/重命名)。
- 机制: **先写日志, 再执行** (WAL, 便于崩溃恢复)。

FSImage (镜像):
- 元数据 **快照** (Checkpoint): 目录树 + 文件映射。
- 作用: 崩溃恢复基准 (HDFS 崩了靠它)。

7. 数据完整性 (Checksum)

机制: 写入时计算 Checksum → 独立存储。

读取: 读时校验 → 失败则 **换副本** 读取。

1. HBase: 基于 HDFS 的大表

核心思想:
- **不依赖 Join**: 把数据视为一张超级大表。
- **切分**: 表切成多个 Region, 分散在不同节点。

优势: 分片清晰, 接近线性扩展 (加机器即扩容)。

2. Column Family (列族)

定义: 列式存储的物理分组。列先归族, 物理独立存储。

设计原则:
- **数量要少**: 建议 **2-3 个**。族多 → 文件多 → 读一行跨多文件 → 慢。
- **族内列**: 可无限多 (稀疏)。
- **语义耦合**: 常一起读写的字段放同族。

3. Schema-less (无模式)

特点:
- **稀疏性**: 不同行可有不同列。
- **存储**: 不存在的列 **不占空间** (不存 Null)。

优势: 适合不规则数据; 无需预留空字段。

4. 表的切分与定位

Region (分区):
- 大表按 **RowKey 范围**水平切分成 Region。
- **自动切分**: 数据增长过大触发 Split。

RegionServer (RS): 管理多个 Region。

定位: 查 Meta / Root 表 (记录 Region 的 RowKey 范围及所在 RS)。

5. 为什么性能行? (读写机制)

写入 (Log-Structure):
- 像 Log 追加, 按时间戳递增 **顺序写**。
- 场景: 适合 "写多、改少"。不擅长频繁随机更新。

读取加速:
- **列族隔离**: 只读相关物理文件。
- **Bloom Filter**: 快速排除不存在的文件。
- **Cache**: BlockCache (热点数据放内存)。

严格一致性: 基于 HDFS 副本机制, 避免读到不一致数据。

6. 存储模型 (Key-Value)

全二进制: RowKey/列名/值均为 `byte[]` (紧凑、好排序)。

Cell: $RowKey + CF : Column + Version$。

多版本 (Version):
- 按 **时间戳 (Timestamp)** 倒序存。
- 默认读最新; 可查历史; 类似 "三维表"。

7. RowKey 设计原则

作用: 决定排序 (字典序)、局部性、查询效率。

原则:
- **唯一性**: 必须唯一。
- **结构**: 业务前缀 + 递增后缀 (时间戳)。

域名反转 (案例):
- 存 `org.apache.www` 而非 `www.apache.org`。
- 目的: 同域名数据 **物理相邻** (局部性) → 范围扫描高效。

8. 时间戳与版本控制

读取规则 (t): 取 $\leq t$ 的最大版本。若无则空。

独立性: 列与列更新时间无关 (不复制旧值)。

空间: 只存变更版本 (节省空间); 最多保留 N 个版本。

9. HBase vs RDBMS

- **RDBMS**: 强 Schema; 支持复杂 Join/索引; 扩展难 (Scale-up); 强事务。
- **HBase**: 弱 Schema (稀疏); **无 Join** (靠 RowKey); 扩展易 (Scale-out/Region 自动分片); 弱事务。
- **RowKey**: 某种意义上像 "聚簇索引"。

10. Hive

定位: HDFS 之上的数据仓库。

面向用户: 熟悉 SQL 的分析师。

本质: 将 **HQL** (类 SQL) 转换为 MapReduce/Spark 任务执行。

1. Flink: Stateful Streaming

定义: 处理流式数据 (Streaming Data) 的计算引擎。

核心定位: **流 + 有状态 (Stateful)**。
- **批处理**: 攒一批 → 统一算 (高延迟)。
- **流处理**: 持续来 → 持续算 (低延迟)。

为什么必须有状态?: 实时计算需依赖历史 (如 WordCount 第 100 次需基于前 99 次结果)。

挑战: 状态存哪? 一致性? 故障恢复? → 引出 Checkpoint。

2. 流类型 时间语义 (Time)

流类型:
- **有界流 (Bounded)**: 有始有终 (本质近批)。
- **无界流 (Unbounded)**: 源源不断 (日志/传感器), Flink 主场。

三种时间 (必考):
1. **Event Time (事件时间)**: 发生时间 (业务语义最准, 需处理乱序)。
2. **Ingestion Time**: 进入 Flink 的时间。
3. **Processing Time**: 算子处理的系统时间 (简单但受抖动影响)。

乱序 (Out-of-Order): 事件发生 ≠ 到达顺序 → 需 Window + Watermark。

3. API 分层: 越底层越强

High (SQL/Table): 声明式 (Group By), 简单但失控。

Mid (DataStream): Map/KeyBy/Window。框架托管状态。

Low (ProcessFunction): 强控 State/Timer/Time。
- **KeyedProcessFunction**: `processElement` (每条触发) + `onTimer` (定时触发)。
- **Timer**: 用于超时报警、状态清理 (防止泄漏)。

4. DataStream Pipeline

典型链路: Map → KeyBy → Window → Reduce。

KeyBy: 核心分流, 同 Key 进入同一并发实例。

Window:
- **Time Window**: 每 30min 聚合。
- **Count Window**: 每 N 条聚合。

5. 运行架构

JobManager (JM): 协调者 (调度、Checkpoint 触发)。

TaskManager (TM): 执行者 (Worker)。

Slot: 并行资源单位 (容器)。

Operator Chain (算子链):
- **链内**: 同线程流水线执行 (减序列化/网络开销)。
- **切链**: 遇到 Shuffle / KeyBy / 重分区。

6. Keyed State Backend

Keyed State: 状态按 Key 分片。KeyBy 后, 不同 Key 分发到不同 Slot, 各 Slot 维护对应 Key 的 State。

State Backend: 状态存哪?
- **Memory**: 堆内存 (快, 怕 OOM)。
- **RocksDB**: LSM-Tree 结构落盘 (适合大状态, 恢复可控)。

7. Checkpoint (CP): 容错核心

机制: 周期性快照 (Snapshot) → 持久化存储。

Barrier (栅栏):
- 特殊标记随流流动。
- 算子收到 Barrier → 触发当前状态快照。

CP 模式:
- **对齐 (Aligned)**: 多输入流时, Barrier 先到的流 **阻塞**等待其他流 Barrier 齐。一致性强, 但反压时易卡死。
- **不对齐 (Unaligned)**: Barrier 不等待。将 Buffer 中 "在路上" 的数据也存入 CP (In-flight data)。吞吐稳, 但 CP 体积大。

8. Watermark (水位线)

定义: 衡量进度的指标。$WM = T$ 表示系统认为 "事件时间 $\leq T$ 的数据已收齐"。

传播机制:
- 随数据流向下游。
- **多输入取 Min**: 下游 WM $= Min(Upstream_WMs)$。最慢的一路决定整体进度。

允许迟到 (Allowed Lateness):
- WM 到达窗口结束时间不立即销毁, 多等一会儿。
- 目的: 修正乱序导致的统计丢失。

结论: WM 不是消灭乱序, 是让乱序 **可控**。

一、总主线

一切安全体系, 最终都绕不开: 非对称加密 (公钥 / 私钥)

二、为什么需要数字签名?

要解决两个核心问题:
1. 身份 (Authentication): ⇒ 这是 "谁" 发的? 是不是他本人?
2. 完整性 (Integrity): ⇒ 内容从发布到接收, 有没有被篡改?

典型场景: 软件发布来源确认、文件防篡改、浏览器证书校验。

三、摘要 (Message Digest / Hash)

1. 摘要是什么?
- 对任意长度数据计算出固定长度的结果。
- 类似 "指纹"。
- 区块链里的区块 Hash 本质也是摘要。

2. 必须满足的性质 (必考)
- 雪崩效应: 原文只改 1 bit, 摘要要完全不同。
- 抗碰撞: 很难构造两段不同消息, 却得到相同摘要。
- ⇒ 摘要解决的是: "是否被篡改"。

3. 遗嘱例子 (完整性)

原文 → 计算摘要 → 保存摘要。改一个名字, 摘要立刻变化。
- **(!)** 问题: 如果保存摘要的人 (律师) 被买通?

四、非对称密钥 (Public/Private Key)

1. 基本特性 (必须背)
- 公钥加密 → 私钥解密
- 私钥加密 → 公钥验证 (在签名语境下, 私钥加密 ≈ 签名)
- 公钥无法反推出私钥。

2. 两种用法 (高频考点)

用法 A: 签名 (身份认证)

私钥签名, 公钥验签。
⇒ 证明: 这东西确实是我发的。

用法 B: 加密通信

用对方公钥加密, 用自己私钥解密。
⇒ 保证: 只有目标接收者能看。

五、数字签名 (Signature)

核心流程 (必写)
1. 发送方: 原文 → 算摘要 → 用私钥对摘要签名。
2. 传输: 原文 + 签名。
3. 接收方: 重新算摘要 → 用公钥验签 → 对比摘要。

解决了什么?

✓ 身份 ✓ 完整性 ✓ 防伪造 ✓ 防事后否认 (律师被买通也没私钥)。

六、公钥分发问题 → 证书

问题本质：“你给我一个公钥，我凭什么信这是你的?”

七、证书与 CA (Certificate Authority)

1. 核心思想

CA 是“大家共同信任的朋友”。

- CA 用自己的私钥 给你的公钥签名 → 得到证书。

2. 验证证书时做什么？

我本地已经信任 CA 的公钥（根证书）→ 用 CA 公钥验证证书签名 → 验证通过 → 信任证书里的公钥。

3. 浏览器报错本质

证书过期 / 域名不匹配 / 证书链断裂 / CA 不可信。

八、Java 体系实践工具

- keytool: 生成密钥对，管理 keystore。
- keystore: 存私钥 + 证书 (alias 标识)。
- 导出证书 (.cer): 含公钥 + 身份信息 + 指纹。
- jarsigner: 给 JAR 包签名 (验证是否被篡改/谁签的)。
- **(!)** 签名 ≠ 加密: 签名是“敢不敢信”，不是“能不能用”。

十、经典组合拳（必考）

非对称 → 安全分发对称密钥；对称 → 高效加密通信。

1. 客户端生成对称会话密钥。
2. 用服务器公钥加密发送。
3. 服务器用私钥解密。
4. 后续全程用对称密钥通信。

十一、SSL / TLS / HTTPS 握手（详解版）

核心定位：在 TCP 握手之后，HTTP 发送之前，协商加密参数。

握手全流程（5 步核心 + 细节）

1. ClientHello (协商发起)
   - **发送**：客户端支持的协议版本 (TLS 1.2/1.3)、加密套件列表 (Cipher Suites)、第一个随机数 (Random_C)。
   - **目的**：告诉服务器“我会什么加密方法，选一个吧”。
2. ServerHello + Certificate (协商确认)
   - **发送**：确认使用的协议版本、选定的加密套件、第二个随机数 (Random_S)、服务器证书 (含公钥 Pub_S)。
   - **目的**：确认加密参数，出示身份证。
3. 客户端验证证书 (关键防伪)
   - **操作**：1. 检查证书有效期、域名是否匹配。2. 用本地受信任的 CA 根证书公钥 解密证书签名，比对摘要。
   - **结果**：验证通过 → 取出证书中的服务器公钥 (Pub_S)。
4. 密钥交换 (Key Exchange)
   - **生成**：客户端生成一个临时随机数，叫 Pre-Master Secret (预主密钥)。
   - **加密**：用服务器公钥 (Pub_S) 加密这个 Pre-Master Secret。
   - **发送**：将加密后的密文发给服务器 (Client Key Exchange)。
   - 此时双方动作：双方利用 $Random_C + Random_S + PreMaster$ 通过算法生成最终的 Master Secret (会话密钥)。
5. 服务器解密 & 握手结束
   - **解密**：服务器用私钥 (Pri_S) 解密拿到 Pre-Master Secret。
   - **生成**：服务器也算出了同样的会话密钥。
   - **验证**：双方发送”Finished”消息（用新会话密钥加密之前的握手摘要），互测能否解密。

为什么需要三个随机数？

$$SessionKey = F(Random_C, Random_S, PreMaster)$$

- **Random_C / Random_S**: 明文传输，防止重放攻击 (Replay Attack)，增加随机性。
- **Pre-Master**: 密文传输，保证密钥的机密性。

十二、Tomcat / Spring Boot HTTPS

- keytool 生成 keystore。
- Tomcat server.xml 配 SSL Connector。
- Spring Boot 配端口 + keystore。
- HTTP → HTTPS 重定向 (8080 → 8443)。
- **建议**：一旦进 HTTPS，尽量全站 HTTPS。

十三、单点登录 (SSO)

目标：登录一次，访问多个系统，不再重复输密码。

十四、Kerberos（重点理解）

核心思想：密码不在网络上传输；用票据 (Ticket) + 会话密钥。

参与方

Client, AS (认证服务器), TGS (票据授予), Service (目标系统)。

两阶段流程（考试描述版）

- 阶段 1 (向认证中心)：客户端发 ID → 认证中心返回：
  - A: 用用户密钥加密的会话密钥
  - B: 用服务密钥加密的票据
- 阶段 2 (访问服务)：客户端带着票据 B + 用会话密钥加密证明 → 服务端解票 → 解密 → 认证成功。

十五、CAS / Kerberos 工程化

认证中心集群、票据存储 (DB/LDAP)、有效期控制。常用于学校/企业统一认证。

十六、安全设计的 6 大目标（必背）

1. 不可否认性
2. 机密性
3. 完整性
4. 可信性 / 双方互防
5. 可用性（防 DDoS）
6. 审计（日志）

十七、纵深防御（三层）

Prevent（防）→ Detect（探）→ Recover（恢）

一、LLM 幻觉 (Hallucination)

1. 一句话结论

幻觉是模型在“不知道答案或理解错误”时，仍然强行生成一个“看起来合理”的答案。

2. 三大来源（必考）

1. 超出知识范围（过期/缺失）
2. 训练语料本身错误
3. 模型理解偏差

二、Prompt Engineering

核心：问得越清楚，模型越稳定；越模糊，越易胡说。

1. 常见技巧

CoT (思维链)、明确角色/限制。

- **作用**：降低幻觉，让输出更“像人思考”。

2. 根本问题（为什么不够用？）

- × 太“玄学”：效果差异大。
- × 模板太长：普通用户写不出几百 token 的 Prompt。
- × 不可规模化：无法要求所有用户都懂 Prompt。

结论：用户只问简单问题 → 系统自动补全上下文 ⇒ **RAG**。

三、RAG (Retrieval-Augmented Generation)

4. 最适合场景（必考）

1. 知识需实时更新：天气、新闻、政策。领域专用知识：校规、内部文档、法律。

四、RAG 技术核心：检索

1. Embedding（文本 → 向量）

2. 文档切块 (Chunking)

- **为什么切？**整篇向量太“粗”；问题只关联局部。
- **怎么切？**自然段、条文、逻辑单元。
- **原则**：1 Chunk ≈ 1 个“可独立理解的最小知识单元”。

3. 向量检索 (Vector Search)

- **注意**：检索的是“上下文”，不是“最终答案”。

总结：**RAG = 检索 + 上下文增强 + 生成**

六、RAG 的质疑与挑战

1. 多模态难

图片/视频/文本向量空间难对齐，相似度比较困难。

2. 切块破坏结构关系（重点）

- Chunk 间可能有因果/流程关系。
- 向量检索只看相似度。
- 后果：搜到“休学流程”，漏掉“复课流程”。

3. 成本爆炸

用户问一句，检索出一大堆 → Token 消耗大，延迟高。

七、架构改进思路

1. 多知识库 (Multi-KB)

不同领域分库，避免混杂。

2. GraphRAG（向量 + 图）

- 向量：找“相似”。
- 图数据库：补“结构关系”。
- **思想**：先语义命中，再结构补全。

3. Agent-based RAG

不预存，通过 Agent 调 API/工具实时获取（股票/天气）。

4. 迭代式上下文扩展

先给少量 Chunk（或摘要），不够再逐步展开。

- **目标**：控制成本 + 保证质量。

一、两大派系：符号 vs 连接

1. 符号主义 (Symbolic AI)

- 核心：知识 = 规则，推理 = 演绎。代表：专家系统。
- 缺陷：规则难以完备，维护难。

2. 连接主义 (Connectionism)

- 核心：知识 = 参数（权重），通过数据训练映射。
- 缺陷：可解释性差 (Black Box)。
- 结论：现为主流，未来趋势是与符号主义融合（可解释/可控）。

二、网络本质与基础设计

1. 学的是什么？

目标是学一个函数 $f(x;\theta)$，让预测 $\hat{y}$ 逼近真实 y。

- 本质：神经网络本质上是函数逼近器，用参数 θ 表示一个可调函数。

2. 为什么必须“非线性”？

- 线性分类只能切直线/超平面，分不开复杂边界。
- 结论：没有非线性激活，多层网络仍等价于一个线性变换，学不出复杂边界。

3. 为什么要“可导”？

- 训练依赖“沿着误差下降最快方向更新”。
- 结论：训练 = 梯度下降；不可导会导致梯度无法计算。

三、神经元与结构

1. 神经元模型 (Perceptron)

公式：$z = \sum w_i x_i + b \to a = \sigma(z)$

- w: 输入重要性；b: 兴奋阈值；σ: 非线性。
- 直觉：神经元是一个“带非线性的特征提取器”。

2. 为什么需要多层？

- 单层只能做简单决策；多层组合出复杂函数。
- 直觉：通过“层层特征变换”，逐步把复杂问题变成更线性可分。

3. 规模与拟合 (Bias-Variance)

- 太小：欠拟合 (Underfitting)。
- 太大：过拟合 (Overfitting，死记硬背)。
- 工程：规模无一刀切，靠验证集调参（炼丹）。

四、数据表示与前向计算

1. 多分类为何用 One-Hot？

- × 单数值分区间：边界敏感，不稳定。
- ✓ One-Hot：类别正交（独立），解释清晰。
- 直觉：把分类变成“选哪个维度最大”，符合概率建模。

2. 矩阵视角与 GPU

- 全连接层：$Z = XW + b$。
- 原因：把“很多神经元/样本的计算”合并成大规模矩阵乘法，GPU 天生擅长并行。

五、核心组件设计 (Why?)

1. Softmax：为什么用 e^z？

公式：$p_k = \frac{e^{z_k}}{\sum e^{z_j}}$

- 指数作用：放大差异，强化最大 Logit。
- 直觉：“拉开差距、强化最大类”，让学习信号更强。

2. 损失函数：为什么用交叉熵 (Cross-Entropy)？

- 强烈惩罚“正确类概率很小”的情况（log 在 0 → 1 陡峭）。
- 直觉：让“错得离谱”代价很大，逼模型把正确概率推向 1。

六、训练流程 (Training Dynamics)

1. 核心三件套

前向 ($\hat{y}$) → 损失 (Diff) → 梯度下降 (Update)。

- $\theta \leftarrow \theta - \alpha \nabla J(\theta)$
- 直觉：训练就是“沿着损失下降最快方向小步走”。

2. 反向传播 (Backpropagation)

- 误差有前后依赖，必须从后往前传。
- 本质：链式法则在计算图上的应用。

七、激活函数演变（必考）

核心原则：让梯度传播更稳定。

1. Sigmoid

- ✓ 平滑，输出 0 ∼ 1 (概率)。
- × 梯度消失：导数最大 0.25，深层链式相乘趋近 0。
- 总结：适合浅层/输出层，深层勿用。

2. Tanh

- 输出 $[-1, 1]$，零中心化，比 Sigmoid 稍好，但仍有梯度消失。

3. ReLU (Rectified Linear Unit)

公式：$\max(0, z)$

- ✓ 计算快；正区间梯度恒为 1（不消失）。
- × Dead ReLU：负区间梯度 0，可能死掉。
- 地位：工程最常用默认选择。

4. 梯度消失 (Vanishing Gradient)

- 直觉：深层网络前几层“学不到东西”，因为反馈回来的误差信号太弱了（被 Sigmoid 杀死了）。

一、为什么全连接 (FC) 不适合图像？

1. 参数爆炸（致命问题）

- 相邻层 100 → 200，参数 2×10^4。图像像素一多，指数级增长。
- 后果：训练成本高，极易过拟合。

2. 根本假设不合理

- FC 假设所有维度同等重要、任意相关。
- 但图像像素有空间位置，邻近关系强于远距离。
- 结论：FC 忽略了空间结构，是一种“全局乱连”的不合理建模。

二、图像识别真正需要什么？

1. 局部性 + 层次结构

- 人类视觉：局部边缘 → 局部形状 → 整体结构。
- 特征分层：小范围 → 大范围；局部 → 全局。
- 结论：图像的本质规律是局部相关 + 层次组合。

三、卷积的直觉 (Convolution)

1. 卷积在干什么？

[Image of convolution operation]

- 滑动窗口 + 局部内积 (加权求和)。
- 卷积核 ≈ 特征探测器：在哪里响应大 → 哪里有该特征。
- 输出矩阵 = 特征图 (Feature Map)。

2. 激活（ReLU）的意义

- 负值置 0，只保留“检测到特征”的位置。
- 作用：降噪 + 稀疏化。

四、为什么要“层层卷”+ 池化？

1. 卷积的层次化

- 第一层边缘 → 第二层形状 → 深层结构。
- 本质：每一层都在扩大感受野。

2. 为什么需要池化 (Pooling)？

- 问题：小核看太少，大核算太慢。
- 解决：下采样缩小特征图 → 特征点“靠近” → 小核也能看大结构。
- 类型：Max Pooling（留最强），Avg Pooling（平滑）。
- 结论：池化用“丢一点细节”换“更大的感受野和更强的不变性”。

五、CNN 优于全连接（考试重点）

1. 关键区别：卷积核是“学出来的”

- 传统：人为设计（边缘/浮雕）；CNN：反向传播自动学。
- 结论：把“特征设计”从人手里交给了数据。

2. 三个核心优势（必背）

1. 局部连接：每个神经元只看一小块。
2. 权重共享：同一个核在整张图复用。
3. 池化降维：降参数、提不变性。

- 本质：CNN 的强大来自“结构先验”，而不是单纯参数多。

六、通道、Padding、Stride 直觉

- 通道 (Channel)：“检测哪一类特征”。输入 RGB=3，输出 = 特征数量。
- Padding：保持尺寸（不 Pad 尺寸会变小）。
- Stride：步长。步子大 → 输出更小。

七、经典结构：LeNet-5 CIFAR

1. LeNet-5 模板

- 卷积（提局部）→ 池化（降维）→ … → 全连接（分类）。
- 结论：省参数主要是因为把“全连接”换成了“卷积 + 共享”。

2. CIFAR-10 训练启示

- 更深 → 表达强 → 准确率高（但训练慢）。
- Takeaway：深度/核大小/通道数是结构设计问题，非数学问题。

八、延伸：卷积与图网络 (GNN)

1. 图计算任务与需求

- 任务：节点分类 / 边预测 / 子图分类。
- 模式往往是结构性的（如洗钱模式）。需从数据学“复杂关系函数”。

2. GNN 两条路线（理解）

1. 谱图理论 (Spectral)：信号处理视角（频率），工程复杂。
2. 消息传播 (Message Passing)：**[主流]**
   -
   - 邻居聚合 → 向外扩散 → 捕捉多跳模式。
   - 类比：CNN 在网格上“卷”，GNN 在邻居集合上“聚合”。

3. 为什么“图上不好卷”？

- CNN 前提：规则网格、固定邻域。
- 图问题：邻居数量不固定、张量形状不一。
- 解决：聚合 (Sum/Mean/Max) 或采样固定数。
- 结论：GNN 关键不是“卷积核”，而是“如何聚合不规则邻居”。

一、CNN 能不能做 NLP？

能，但有边界。

1. 1D 卷积思想

- 图像是二维卷积（高 × 宽），文本是一维卷积（序列方向滑动）。
- 前提：输入必须是“规则结构 + 局部模式有意义”的数据。

二、NLP 基石：Embedding

1. One-hot（为什么不用）

- 高维、稀疏、正交（无法表达语义相似）。
- 否定：只表示“是不是这个词”，不表示“像不像另一个词”。

2. Dense Embedding（必考）

- 低维稠密向量，距离反映语义相似性。
- 本质：Embedding 把“离散符号”映射到“连续语义空间”。

三、CNN 在 NLP 中的机制

- 核心：句子 = Embedding 序列。
- 操作：卷积核滑动，每次看 n 个词 (n-gram)。
- 结论：CNN 在 NLP 中学的是局部 n-gram 模式 (如 kernel=4 学 4-gram)。

四、CNN 做 NLP 的根本缺陷

- 固定窗口：语言结构灵活，卷积核长度死板。
- 长距离依赖弱：远距离词难以直接联系，需多层传递。
- 过渡：CNN 擅长局部组合，但不擅长建模长程依赖 → 引出 RNN。

五、RNN：把“历史”带到当前

1. 核心思想

- 公式：当前状态 = 当前输入 + 过去状态的函数。
- h_t 是对“到目前为止序列”的摘要。
- 直觉：看一个词时，不是孤立看，而是“带着上下文看”。

2. 优势

- 天然适合序列、句子长短可变。

六、RNN 应用：时间序列预测

- 不仅是文本，还有股票、天气、传感器。
- 本质：RNN 是“有记忆的函数逼近器”。

七、RNN 致命伤：长期依赖

1. 现象

- 早期信息一直往后传，被稀释，没有“主动忘记”机制。

2. 后果（训练层面）

- 梯度消失 / 爆炸：学不到远距离依赖。
- 过渡：语言需要“记住重要的、忘掉无关的”，普通 RNN 做不到 → 引出 LSTM。

八、LSTM：用门控机制“控制记忆”

1. 核心思想

不是简单传状态，而是“选择性地记、选择性地忘”。

2. 三大门（必背结构）

1. 遗忘门 (Forget)：决定过去的信息留多少（句号/换段 → 忘）。
2. 输入门 (Input)：决定当前新信息写多少。
3. 输出门 (Output)：决定当前对外输出多少。

3. 细胞状态 (Cell State)

- 像一条“主干记忆通道”，信息平滑流动，缓解梯度消失。
- 本质：LSTM 学会“什么时候该记、什么时候该忘”，缓解长期依赖。

九、工程 Demo 启示

- 时序预测：滑动窗口 → 监督学习。简单规律下，复杂模型未必更好。
- 文本生成：Softmax 做概率分布，预测 = 条件概率建模。
- 注：*真正强的生成要靠 Transformer。*

十、CNN vs RNN（高频简答）

CNN	RNN / LSTM
擅长：空间局部结构	擅长：时间序列依赖
关键词：局部特征，权重共享，分层抽象	关键词：状态传递，上下文记忆，门控
任务：图像，文本 n-gram	任务：文本，时序预测

一句话对比：CNN 建模“空间局部相关”，RNN 建模“时间序列相关”。

十一、多模态结合 (Multimodal)

- 为什么要结合？
- 图像 (空间) → CNN 提取特征。
- 文本 (序列) → RNN/LSTM 生成描述。
- 典型任务：Image Caption (看图说话)，VQA (视觉问答)。
- 设计：不同网络各司其职，通过接口拼起来。

0. 为什么要讲 Transformer？

RNN/LSTM 虽能处理序列，但长链条信息会被“门控乘法/梯度传播”稀释。Transformer 用自注意力 (Self-Attention) 让任意位置直接建立联系，解决长依赖的“路径过长”问题。

1. 从 RNN 的瓶颈出发

RNN/LSTM 结构性缺陷

- 串行：一个词一个词处理，展开是长链条。
- 稀释：信息传递步数多 → 越早的信息越模糊。
- 训练：梯度消失/爆炸 → 长程规律学不稳。
- 本质：长期依赖的本质是“信息/梯度要走很长的路”，路越长越衰减。

2. Transformer 核心思想

- 输入端：并行处理整段序列 (Parallel)。
- 计算：Token 之间并行计算关系（不再递归）。
- 输出端：仍是自回归 (Autoregressive) 逐个生成。

3. 结构总览：Encoder + Decoder

1. Encoder (理解输入)：
   - 把句子编码成上下文特征。可堆叠 (经典 6 层)。
2. Decoder (生成输出)：
   - 逐 Token 生成。
   - 看已生成部分 (Masked Self-Attn) + 看 Encoder 输出 (Cross-Attn)。

一句话：Encoder 负责“读懂”，Decoder 负责“按条件写出来”。

4. Layer 内部模块（必背）

- Encoder Layer: Multi-Head Self-Attention + FFN。
- Decoder Layer: Masked Self-Attention (防看未来) + Cross-Attention (对齐信息) + FFN。

5. 为什么 Attention 后必接 FFN？

- Attention (QK V) 本质是线性运算 (矩阵乘法)。
- 线性层叠加 ≡ 一个线性层 → 深度白堆。
- FFN: 引入非线性 (ReLU/GELU) → 提升表达力。
- 金句：FFN 作用是“给注意力后的表示加入非线性”，否则深度没有意义。

6. Attention 机制：Q / K / V

1. 直觉解释

- Q (Query)：我想找什么。
- K (Key)：我有什么标签/索引。
- V (Value)：真正的信息内容。
- 本质：Attention = “按相关性从一堆信息里动态取用”。

2. 为什么要除以 $\sqrt{d_k}$ (Scaling)？

- 维度大时点积数值变大 → Softmax 变尖 (饱和)。
- 后果：梯度消失，训练不稳定。
- 目的：把分布拉回可训练区间，解决梯度问题。

7. Self-Attention：句内关联

- Q/K/V 来自同一句。解决指代 (如“它”指谁)。
- 输出不再是“词本身”，而是“上下文融合后的词”。

8. Multi-Head（多头）为什么必要？

- 动机：单头 = 单视角。语言关系复杂 (语法/语义/指代)。
- 机制：多子空间并行观察 → 合并。
- 模板：Multi-Head 把注意力拆成多个“观察角度”，不同 Head 学不同关系，提升表达力和鲁棒性。

9. 位置编码 (Positional Encoding)

- Attention 天然把输入当“集合”(Set)，不懂顺序。
- 必须注入“顺序/距离信息”(Embedding + Pos Encoding)。
- 模板：解决“注意力不含顺序”缺陷，使模型同时利用内容相关性 + 位置结构。

10. Decoder Mask（防作弊）

- 生成第 t 个词时，只能看 $1 \ldots t-1$。
- Mask：将未来位置得分设为 $-\infty$ (Softmax 后为 0)。
- 目的：保证生成“只基于过去”，符合因果性。

11. Transformer 做时序预测

- 迁移：时间点 = 位置，数值 =Embedding。
- 架构：只用 Encoder 提特征 → 接 MLP 回归。
- 本质：Transformer Encoder = 通用序列特征提取器。

12. 必考问答模板（背这 4 个）

Q1：为什么能缓解长期依赖？

1. 全局直连：路径 $O(n) \to O(1)$，梯度不衰减。
2. 动态选择：按相关性直接关注远处重要 Token。
3. 并行训练：不依赖递归，优化更稳。

Q2：为什么要位置编码？

- Attention 置换不变 (不懂顺序)。
- 注入“第几个/相对距离”信息，让模型学到语序和指代。

Q3：多头注意力解决了什么？

- 单头视角单一。
- 多头在不同子空间并行建模 (语法/语义/局部/全局)。
- 融合后表示更丰富、鲁棒。

Q4：电子书店项目怎么用？(落地)

1. 文本类：

- 评论检索：Embedding → 向量检索 (解决同义词)。
- 摘要：长评摘要，提升效率。

2. 时序类：

- 销量预测：历史销量/节假日序列 → Encoder → 回归。
- 库存预警：预测销量 + 补货周期。

Part 1: Large Language Models (LLM)

A. 大模型最核心一句话（必背）

- LLM 每一步只做一件事：给定上文，预测下一个 token。
- 自回归 (Autoregressive)：预测 → 拼回上下文 → 再预测 → 直到 EOS。

B. “相似性”从哪来？

- 文本 → 向量 (Embedding)。
- 模型倾向于在向量空间匹配“类似上下文模式”。
- 本质：在向量空间里做模式匹配，再输出概率分布。

C. 温度 (Temperature)

- Argmax：每次取最大概率 → 答案固定，缺多样性。
- Temperature：控制“随机采样程度”。
- 低温 (→ 0)：保守、像检索；高温：随机、创造性。
- 本质：不是让模型更聪明，而是调“确定性 vs 多样性”的权衡。

D. Token 越多为何越慢？

- 生成第 t 个词，需把前 $t-1$ 个当上下文参与计算。
- 本质：上下文窗口决定能“记住”多长，也决定计算开销 (复杂度 $O(N^2)$)。

E. 从 N-gram 到大模型（必考逻辑）

为什么必须用神经网络？

1. N-gram 直觉：Bigram 统计“前一个 → 后一个”概率表。
2. 查表爆炸：词表 4 万，Bigram 表 $4W \times 4W$；3-gram 直接崩。
3. LLM 的解法：神经网络逼近那张无法存储的巨大概率表。

Part 2: Reinforcement Learning (RL)

F. RL 最直观一句话

RL = 智能体在环境里试错，通过奖励/惩罚学会策略。

G. 形式化框架 (MDP)

- 交互：$S_t \to A_t \to R_{t+1} \to S_{t+1}$。
- 观察：可能是局部 (马里奥) 或全局。
- 轨迹 (Trajectory)：一串 S-A-R 序列。
- Episodic：有终点 (游戏) vs Continuing：无终点 (交易)。

H. Return + 折扣因子 γ

- 目标：最大化长期累积收益。
- γ (Discount Factor)：控制看重未来的程度 (0.99 重视长期)。
- 本质：“未来收益不确定，所以要打折”。

I. 探索 vs 利用 (ϵ-greedy)

- 利用 (Exploit)：选当前最优。
- 探索 (Explore)：试错找更优。
- ϵ-greedy：$1-\epsilon$ 概率选最优，ϵ 概率随机。
- 本质：不探索陷局部最优，不利用学不到稳定策略。

J. Policy-based vs Value-based

1. Policy ($\pi(a|s)$)：直接输出动作/概率。
2. Value ($Q(s,a)$)：算每个动作的“价值”，选分最高的。

一句话：Policy 告诉你“做什么”，Value 告诉你“哪个更值”。

K. Q-Learning DQN

1. Q-table（小规模）

- 行 S，列 A，表项 $Q(s,a)$。状态多时不可行。

2. DQN (Deep Q-Network)

- 用神经网络近似 $Q(s,\cdot)$。输入 S，输出所有 A 的值。
- 本质：DQN = “Q-table 的函数逼近版”。

L. Q-learning 更新公式

$$Q(s,a) \leftarrow Q(s,a) + \alpha\big(\underbrace{r + \gamma \max Q'}_{\text{Target}} - \underbrace{Q(s,a)}_{\text{Current}}\big)$$

- Target：真实奖励 + 估计的未来最好结果。
- Correction：用 TD 误差做小步修正 (α)。
- 本质：不断把当前的估计拉向更准的目标值。

N. 算法结构与超参

- 结构：外层循环 (Episode 玩很多局) + 内层循环 (Step 玩每一步)。
- α (学习率)：步子大小。
- γ (折扣)：远见程度。
- ϵ (探索)：冒险精神。